

1. УВОД И ПОСТАВКА ЗАДАТКА

1.1. Циљ задатка

Циљ овог задатка јесте пројектовање, рачунарска симулација и анализа рада дигиталних П (Пропорционалног) и ПИ (Пропорционално-Интегралног) регулатора у систему са затвореном повратном спрегом. Управљани објекат представља једносмерни електромотор (односно процес првог реда) дефинисан континуалном преносном функцијом првог реда која износи:

$$G(s) = \frac{K}{\tau s + 1} = \frac{0.8}{0.3 s + 1}$$

Где је $K = 0.8$ статичко појачање објекта, а $\tau = 0.3$ [s] временска константа система. Кроз практичну имплементацију у окружењу Siemens TIA Portal (SCL) и визуелизацију на HMI Trend View графику, циљ је верификовати следеће објективне показатеље квалитета управљања:

- Поређење П и ПИ режима: Анализа утицаја пропорционалног појачања K_c на брзину одзива и доказивање присуства статичке грешке у стационарном стању при примени само П регулатора.
- Елиминација статичке грешке: Показивање како увођење интегралног дејства (K_i) потискује стационарну грешку на нулу.
- Спречавање засићења интегратора (Anti-Reset Windup): Практична верификација алгорита за ограничење интегралне суме у условима засићења (односно сатурације) управљачког сигнала.

1.2. Одређивање почетних параметара и дискретизација

Да би се континуални алгоритам могао реализовао у PLC управљачком блоку OB30 (Cyclic Interrupt) са периодом одабирања $T_s = 0.04$ [s] (односно 40 [ms]), извршена је дискретизација објекта и прорачун параметара регулатора.

Применом ЗОН принципа дискретизације (или аналитичким решењем диференцијалне једначине за период $T_s = 0.04$ [s] и $\tau = 0.3$ [s]), добија се релација којом се у SCL коду у реалном времену израчунава излаз објекта $y[k]$, на следећи начин:

$$e^{-\frac{T}{\tau}} = e^{-\frac{0.04}{0.3}} = e^{-0.13333} \approx 0.8751733$$

$$K * \left(1 - e^{-\frac{T}{\tau}}\right) = 0.8 * (1 - 0.87517) = 0.8 * 0.1248267 \approx 0.0998613$$

$$y[k] = e^{-\frac{T}{\tau}} * y[k-1] + K \left(1 - e^{-\frac{T}{\tau}}\right) * u[k-1]$$

$$y[k] = 0.8751733 * y[k-1] + 0.0998613 * u[k-1]$$

Структурне шеме отворене и затворене спреге и дефиниција свих сигнала дате су у делу 1.3.

Структура овог система представља дигитални систем са затвореном повратном спрегом (односно управљачку петљу у реалном времену). Елементи структуре и њихови називи су следећи:

- Задата вредност (референтна вредност) $w[k]$ или $r[k]$, која представља жељену вредност излаза (у нашем случају одсечна побуда $w = 100$).
- Грешка праћења $e[k]$, се дефинише као разлика између задате и стварне вредности, односно $e[k] = w[k] - y[k]$. Њена вредност по завршетку прелазног процеса представља грешку стационарног стања e_{st} .
- Управљачки сигнал $u[k]$ јесте излаз из регулатора који се шаље према објекту
- Управљани објекат $G(s) / G(z)$, јесте једносмерни електромотор (процес првог реда) са преносном функцијом $G(s) = K/(ts+1) = 0.8/(0.3s + 1)$.
- Излазна (управљана) величина $y[k]$ јесте физичка величина која се мери и регулише (брзина обртања мотора).
- Извршни орган и дискретизатор (ZOH) јесте извршни елемент са колом за задржавање нултог реда (Zero-Order Hold) који континуирани објекат повезује са дискретним ПЛЦ-ом.

ПИ (Пропорционално-интегрални) регулатор је дискретни управљачки алгоритам чији се излазни сигнал $u[k]$ састоји од две компоненте, пропорционалног дејства ($u_p[k] = K_c * e[k]$) које реагује тренутно и пропорционално тренутној грешци, обезбеђује брзину одзива, али када ради сам за себе (у П-режиму) оставља статичку грешку у стационарном стању, и интегралног дејства ($u_i[k]$) које акумулира грешку кроз време. Његова главна улога је да потпуно елиминира статичку грешку у стационарном стању, тако да излаз $y[k]$ у потпуности достигне задату вредност $w[k]$ ($e_{st} = 0$).

Основни технички услови (критеријуми квалитета управљања) које овај систем мора да испуни су:

- Тачност у стационарном стању, односно потпуна елиминација статичке грешке ($e_{st} = 0$), тј. $y_{st} = w$.
- Брзина одзива и стабилност, постизање што краћег времена смирења (t_s) без осцилација и без прескока ($S_p = 0$).
- Ограничење управљачког сигнала, спречавање прекорачења физичких граница опреме ($0 \leq u[k] \leq 125$) уз спречавање појаве сатурације интегратора помоћу Anti-Windup механизма.

Подешавање параметара ПИ регулатора спроведено је методом поништавања доминантног пола објекта (pole-zero cancellation), где се интегрална временска константа регулатора T_i изједначава са временском константом објекта τ , односно:

$$G_R(s) = K_c * \left(1 + \frac{1}{T_i s}\right) = K_c * \frac{T_i s + 1}{T_i s}$$

Узимајући у обзир преносну функцију управљаног објекта првог реда:

$$G(s) = \frac{K}{\tau s + 1}$$

Преносна функција отворене директне гране система $G_{otv}(s) = G_R(s) * G(s)$ добија облик:

$$G_{otv}(s) = \left(K_c * \frac{T_i s + 1}{T_i s}\right) * \left(\frac{K}{\tau s + 1}\right)$$

$$G_{otv}(s) = K_c * \frac{T_i s + 1}{T_i s} * \frac{K}{\tau s + 1} = \frac{K_c * K}{T_i s}$$

$$T_i = \tau = 0.3 \text{ s}$$

На основу релације за континуално интегрално појачање $K_i = K_c/T_i$ и за усвојено пропорционално појачање у ПИ режиму $K_c = 3$, израчунава се континуално интегрално појачање:

$$K_i = \frac{K_c}{\tau} = \frac{3.0}{0.3 \text{ s}} = 10.0 \text{ s}^{-1}$$

Пошто се у дискретном SCL коду интегрална сума ажурира у сваком кораку одабирања $T_s = 0.04 \text{ [s]}$, у коду се рачуна дискретни корак интеграције K_{i_step} :

$$K_{i_step} = K_i * T = 10.0 * 0.04 = \mathbf{0.4}$$

Објекат је дискретизован егзактном ZOH дискретизацијом, док се интегрални члан регулатора реализује Ојлеровом апроксимацијом уназад: $u_i[k] = u_i[k-1] + K_i \cdot T_s \cdot e[k]$. Границе управљања су $u_{min} = 0$ и $u_{max} = 125$, па је $y_{max} = K \cdot u_{max} = 0.8 \cdot 125 = 100$.

Будући да је овај рад заправо наставак задатка 1, као коришћени контролер се поново бира Simatic S7-1200 CPU, и то тачније модел CPU 1214C DC/DC/DC, верзија 4.4.

1.3. Структура система и дефиниција сигнала

Пре описа PLC имплементације дефинишу се структура система и значење свих сигнала. Разматрају се два режима рада истог дискретног модела објекта $G_d(z)$: отворена спрега и затворена повратна спрега. Све величине су дискретне, у тренуцима $k \cdot T_s$, где је период одабирања $T_s = 0.04$ [s] (циклични прекид OB30).

- Модел објекта

Континуални објекат $G(s) = K/(\tau s + 1) = 0.8/(0.3s + 1)$ дискретизован је егзактно ZOH поступком (Zero-Order Hold) за период $T_s = 0.04$ [s]:

$$G_d(z) = b/(z - a), \quad a = e^{-T_s/\tau} \approx 0.87517, \quad b = K(1 - a) \approx 0.09986$$

Односно $y[k] = a \cdot y[k-1] + b \cdot u[k-1]$. Ова једначина се извршава у блоку FB_Plant; коефицијенти у SCL коду заокружени су на пет децимала.

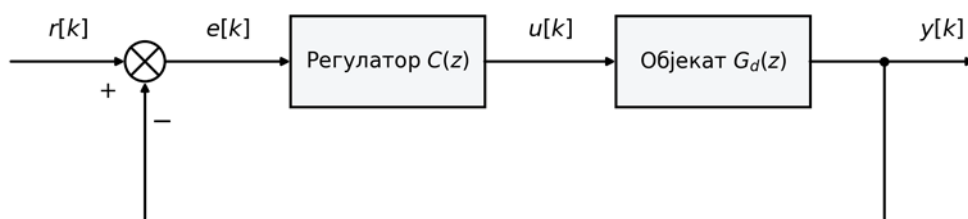
- Објекат у отвореној спрези

У отвореној спрези нема регулатора: улазни сигнал $u_{OL}[k]$ задаје се директно на улаз објекта (у симулацији је то одскачна побуда $u_{OL}[k] = 100$).



- Систем са затвореном повратном спрегом

У затвореној спрези референтни сигнал $r[k]$ пореди се са излазом $y[k]$. Регулатор $C(z)$ из грешке праћења $e[k] = r[k] - y[k]$ формира управљачки сигнал $u[k]$, који делује на објекат.



- Регулатор

У П режиму је $u[k] = u_P[k] = K_C \cdot e[k]$. У ПИ режиму је $u_P[k] = K_C \cdot e[k]$, а интегрална компонента се ажурира текућом грешком (Ојлерова апроксимација уназад):

$$u_I[k] = u_I[k-1] + K_I \cdot T_s \cdot e[k], \quad u_I[k] \in [0; 125], \quad u[k] = u_P[k] + u_I[k]$$

Излаз регулатора се на крају сатурише на $0 \leq u[k] \leq 125$. Усвојени параметри, у П режиму $K_c = 5$; у ПИ режиму $K_c = 3$ и $K_i = 10 \text{ s}^{-1}$, чиме је $T_i = K_c/K_i = 0.3 \text{ [s]} = \tau$ (поништавање пола објекта).

- Сигнали и одговарајуће PLC променљиве

Значење сваког сигнала и његова веза са PLC таговима дати су у табели 1.

Величина	Ознака	PLC tag	Напомена
Референтни сигнал (задата вредност)	$r[k] (= w[k])$	Referentna_vrednost	Само у затвореној спрези; HMI поље за унос
Улаз објекта у отвореној спрези	$u_{ol}[k]$	Referentna_vrednost	Исто HMI поље; у FB1: $u_{out} := w_setpoint$
Грешка праћења	$e[k] = r[k] - y[k]$	greska_pracenja	Тренутна грешка; у отвореној спрези се не рачуна ($e_{out} = 0$)
Управљачки сигнал (излаз регулатора)	$u[k]$	matemacko_upravljanje	Улаз објекта у затвореној спрези; $0 \leq u \leq 125$
Излаз објекта	$y[k]$	izlazna_vrednost_objekta	Излаз дискретног модела FB_Plant
Пропорционална компонента	$u_p[k]$	proporcionalno_dejstvo	$u_p[k] = K_c \cdot e[k]$
Интегрална компонента	$u_i[k]$	vrednost_integrالا	Ограничена на $[0; 125]$
Грешка стационарног стања	e_{st}	/	$e_{st} = \lim_{k \rightarrow \infty} e[k]$; очитава се из Trace-а по завршетку прелазног процеса

Табела 1: Значење и ознака сигнала у софтверу

- Грешка праћења и грешка стационарног стања

Величина $e[k] = r[k] - y[k]$ је тренутна грешка праћења и током прелазног процеса није једнака нули. Грешка стационарног стања e_{st} јесте њена вредност по завршетку прелазног процеса, $e_{st} = \lim_{k \rightarrow \infty} e[k]$. За $r = 100$ очекује се: у П режиму ($K_c = 5$) $y_{st} = K_c K / (1 + K_c K) \cdot r = 80$, односно $e_{st} = 20$ и $u_{st} = 100$; у ПИ режиму $e_{st} = 0$.

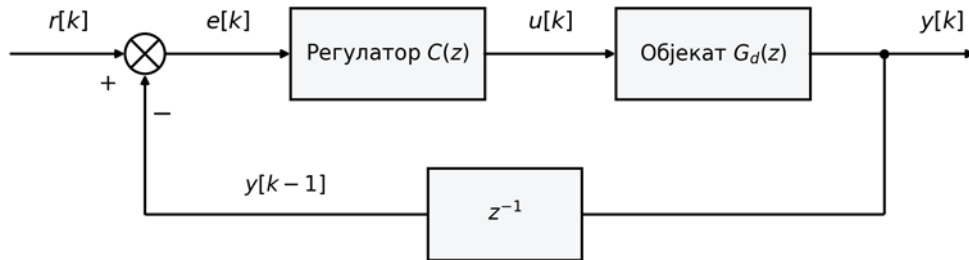
- Границе управљања и опсег референце

Ограничење $0 \leq u \leq 125$ примењује се на излаз регулатора и на стање интегратора; сам објекат нема сатурацију. Излаз је ограничен само посредно, $y_{max} = K * u_{max} = 0.8 * 125 = 100$, па је опсег референце $0 \leq r \leq 100$. Вредност $u_{ol} = 100$ у тесту отворене спреге је амплитуда побуде, а не граница. Референца $r = 100$ захтева $u_{st} = 125$, односно управљање на самој граници сатурације.

- Редослед извршавања у OB30

У OB30 се FB1 (регулатор) позива пре FB_Plant (објекат) програмског блока. Регулатор зато рачуна грешку из излаза објекта у претходном циклусу, $e[k] = r[k] - y[k-1]$. Због тога петља у PLC реализацији, поред кашњења од једног такта у $G_d(z)$, садржи и један такт

кашњења у повратној грани (блок z^{-1}). Овај ефекат објашњава мали прескок излаза у П режиму (максимум 81.37 при $y_{st} = 80$) и биће укључен у Simulink модел који се користи за верификацију касније.



2. ДЕФИНИСАЊЕ И АНАЛИЗА PLC ПРОМЕНЉИВИХ (PLC TAGS)

За исправно функционисање математичког модела, алгоритама управљања и комуникације са HMI панелом, дефинисана је табела глобалних PLC променљивих (Default tag table). Све променљиве су Real типа (осим управљачких битова) како би се омогућила обављање прецизних прорачуна са добијеним вредностима током симулације.

Комплетирана табела PLC тагова треба да изгледа као следећа табела:

	referentna_vrednost	Real	%MD10	☐	☒	☒	☒
	izlazna_vrednost_objekta	Real	%MD14	☐	☒	☒	☒
	matemacko_upravljanje	Real	%MD18	☐	☒	☒	☒
	greska_pracenja	Real	%MD30	☐	☒	☒	☒
	vrednost_integrالا	Real	%MD22	☐	☒	☒	☒
	izbor_rezima_regulatora	Bool	%M0.1	☒	☒	☒	☒
	mod_motora	Bool	%M0.0	☒	☒	☒	☒
	proporcionalno_dejstvo	Real	%MD26	☐	☒	☒	☒
	Kc_P	Real	%MD34	☐	☒	☒	☒
	Kc_PI	Real	%MD38	☐	☒	☒	☒
	Ki_step	Real	%MD42	☐	☒	☒	☒
	izbor_zadatka	Bool	%M0.2	☒	☒	☒	☒
	Kc_HMI_input	Real	%MD46	☐	☒	☒	☒
	Ki_HMI_input	Real	%MD50	☐	☒	☒	☒

“referentna_vrednost”: Референтна вредност коју корисник уноси преко HMI-ја (жељена вредност излаза). У затвореној спрези то је референца $r[k]$, а у отвореној спрези непосредни улаз објекта $u_{ol}[k]$

“izlazna_vrednost_objekta”: Тренутна излазна вредност из симулатора објекта (y) која се приказује на HMI графику.

“matemacko_upravljanje”: Укупно израчунати управљачки сигнал (u) који регулатор шаље објекту управљања.

“greska_pracenja”: Грешка праћења $e[k] = r[k] - y[k]$ код затворене спреге, код отворене спреге грешка праћења се не рачуна.

“vrednost_integrала”: Акумулирана вредност интегралног дејства (u_i) унутар ПИ регулатора.

“izbor_rezima_regulatora”: Дугме на HMI-ју за избор режима рада регулатора (FALSE = П режим, TRUE = ПИ режим).

“mod_motora”: Главни прекидач на HMI-ју за укључивање/исључивање рада система (START/STOP дугме).

“proporcionalno_dejstvo”: Израчуната вредност пропорционалне компоненте управљања ($u_p = K_c * e$).

“Kc_P”: Пропорционално појачање које се примењује када је регулатор у П режиму.

“Kc_PI”: Пропорционално појачање које се примењује када је регулатор у ПИ режиму.

“Ki_step”: Израчунати дискретни корак интеграције ($K_{i_step} = K_i * T_s$) за период одабирања од 40 [ms].

“izbor_zadatka”: Дугме на HMI-ју за избор структуре управљања (FALSE = затворена повратна спрега, TRUE = отворена петља).

“Kc_HMI_input”: Поље на HMI-ју преко којег оператер задаје пропорционално појачање K_c .

“Ki_HMI_input”: Улазно поље на HMI-ју преко којег оператер задаје интегрално појачање K_i .

Како би детаљно објаснили улоге променљивих, можемо их сврстати у четири групе:

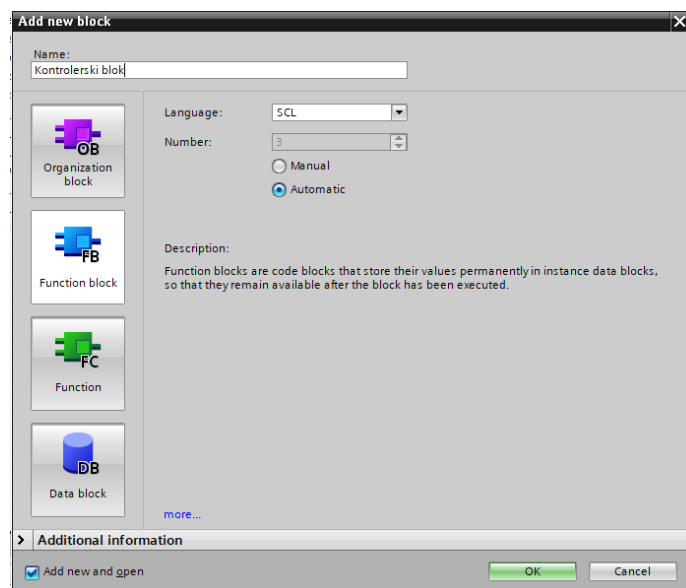
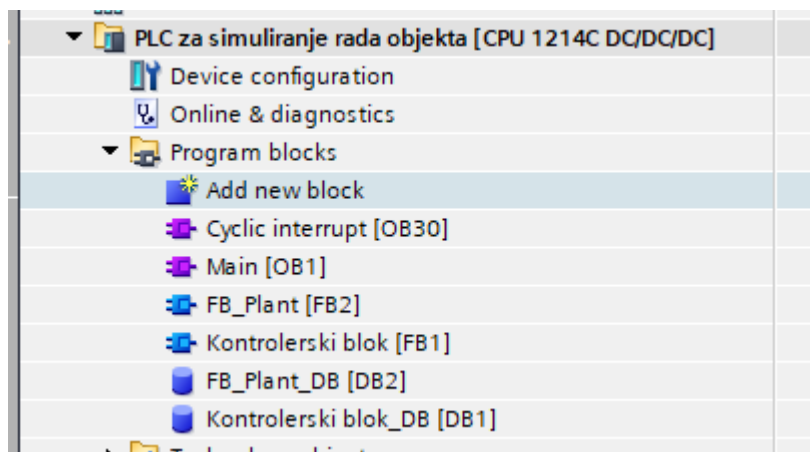
- Управљачки битови (mod_motora, izbor_rezima_regulatora, izbor_zadatka), служе за избор режима рада и управљање симулацијом. mod_motora омогућава ресетовање свих променљивих на 0 када је мотор искључен, izbor_rezima_regulatora преусмерава извршавање SCL кода између П и ПИ алгорита, док izbor_zadatka омогућава избор између затворене повратне спреге и отворене петље.
- Улазне и излазне процесне променљиве (referentna_vrednost, izlazna_vrednost_objekta, greska_pracenja), чине основну петљу са повратном спрегом. У сваком циклусу од 40 [ms], сигнал грешке greska_pracenja се прерачунава као разлика задате вредности referentna_vrednost и тренутног одзива мотора izlazna_vrednost_objekta.
- Компоненте управљања (proporcionalno_dejstvo, vrednost_integrала, matematicko_upravljanje), proporcionalno_dejstvo представља тренутно дејство сразмерно грешци, док vrednost_integrала акумулира претходна одступања ради елиминације статичке грешке. matematicko_upravljanje формира коначни сигнал који делује на дискретни модел мотора.
- Параметри регулатора (K_c , K_c , K_i , K_c , K_i), нам омогућавају независно подешавање појачања за различите режиме рада. Раздвајањем K_c за П и ПИ режим омогућено нам је да посматрамо пропорционално појачање у случају деловања обе врсте регулатора.

У HMI интерфејсу и SCL блоку предвиђен је један улазни параметар појачања K_c , чија се вредност динамично прослеђује регулатору у зависности од изабраног режима рада (П или ПИ). У П режиму је $K_c = 5$, а у ПИ режиму $K_c = 3$ (различите вредности су намерне). K_i се уноси независно, а поништавање пола ($T_i = K_c/K_i = \tau = 0.3$ [s]) важи само за $K_c = 3$ и $K_i = 10 \text{ s}^{-1}$.

3. ИМПЛЕМЕНТАЦИЈА У СИСТЕМУ TIA PORTAL (SCL)

3.1. Креирање и програмирање контролерског блока

Након креирања табеле глобалних тагова (Default tag table), приступа се изради првог кључног дела пројекта– контролерског блока (FB1). Овај блок је реализован као функцијски блок (Function Block – FB) у SCL језику, чиме је омогућена његова потпуна реализованост и независност од остатка кода. Па тако:



Пре писања саме управљачке логике, дефинисан је интерфејс блока који се састоји од улазних, излазних и статичких променљивих које се уносе на следећи начин:

Kontrolerski blok									
	Name	Data type	Default value	Retain	Accessible f...	Writa...	Visible in ...	Setpoint	
1	Input								
2	enable	Bool	false	Non-ret...	☒	☒	☒	☐	
3	open_loop	Bool	false	Non-retain	☒	☒	☒	☐	
4	pi_mode	Bool	false	Non-retain	☒	☒	☒	☐	
5	w_setpoint	Real	0.0	Non-retain	☒	☒	☒	☐	
6	y_actual	Real	0.0	Non-retain	☒	☒	☒	☐	
7	Kc_in	Real	0.0	Non-retain	☒	☒	☒	☐	
8	Ki_in	Real	0.0	Non-retain	☒	☒	☒	☐	
9	Output								
10	u_out	Real	0.0	Non-retain	☒	☒	☒	☐	
11	e_out	Real	0.0	Non-retain	☒	☒	☒	☐	
12	u_p_out	Real	0.0	Non-retain	☒	☒	☒	☐	
13	u_i_out	Real	0.0	Non-retain	☒	☒	☒	☐	
14	Kc_P_out	Real	0.0	Non-retain	☒	☒	☒	☐	
15	Kc_PI_out	Real	0.0	Non-retain	☒	☒	☒	☐	
16	Ki_step_out	Real	0.0	Non-retain	☒	☒	☒	☐	
17	InOut								
18	<Add new>								
19	Static								
20	u_integral_stat	Real	0.0	Non-retain	☒	☒	☒	☐	
21	Tempo								

Улазни параметри (под групом Input):

- enable (Bool): Дозвола рада регулатора (активира се преко тага mod_motora).
- open_loop (Bool): Избор структуре управљања (FALSE = затворена повратна спрега, TRUE = отворена петља).
- pi_mode (Bool): Избор режима рада регулатора (FALSE = П режим, TRUE = ПИ режим).
- w_setpoint (Real): Задата вредност (референтни сигнал са HMI-ја $r[k]$ у случају управљања у затвореној спрези, а улаз објекта $u_{ol}[k]$ у случају управљања у отвореној спрези).
- y_actual (Real): Тренутни одзив/излаз објекта.
- Kc_in (Real): Пропорционално појачање унето са HMI-ја (Kc_HMI_input).
- Ki_in (Real): Интегрално појачање унето са HMI-ја (Ki_HMI_input).

Излазни параметри (под групом Output):

- u_out (Real): Укупно израчунати управљачки сигнал који се прослеђује објекту.
- e_out (Real): Грешка праћења $e[k] = r[k] - y[k]$ (у отвореној спрези $e_out = 0$).
- u_p_out (Real): Пропорционална компонента управљачког сигнала.
- u_i_out (Real): Интегрална компонента управљачког сигнала.
- Kc_P_out (Real): Вредност појачања K_c за П режим.
- Kc_PI_out (Real): Вредност појачања K_c за ПИ режим.
- Ki_step_out (Real): Израчунати дискретни корак интеграције (Ki_step).

Статичке променљиве (под групом Static):

- u_integral_stat (Real): Интерни акумулатор интегралне суме који чува вредност између два циклуса одабирања.

Алгоритам унутар контролерског блока извршава следеће операције:

1. Провера дозволе рада (enable), Ако је enable = FALSE, све излазне променљиве и интерна сума u_integral_stat ресетују се на 0.0.
2. Прорачун грешке и дискретног корака, у затвореној петљи грешка праћења рачуна се као $e = w - y$; у отвореној петљи регулатор се не користи: $e_out = 0$, а $u_out = w_setpoint$ (референтно поље је тада непосредни улаз објекта $u_{ol}[k]$).
3. Дискретни корак интеграције ажурира се на основу унетог интегралног појачања: $Ki_step = K_i * T_s = K_i * 0.04$. Ki_step_out се рачуна само у ПИ грани.
4. Формирање управљања (П или ПИ режим), за П режим ($pi_mode = FALSE$) рачуна се само пропорционално дејство $u_p = K_c * e$, док се u_integral_stat ресетује на 0.0. За ПИ режим ($pi_mode = TRUE$) акумулира се интегрална сума а укупни сигнал формира се као збир пропорционалног и интегралног дејства ($u = u_p + u_i$). Интегрална сума се ажурира текућом грешком (Ојлер уназад), $u_integral_stat = u_integral_stat + Ki_step * e$, и ограничава на $[0.0; 125.0]$ (state clamping). На крају се укупно управљање u ограничава на опсег $[0.0; 125.0]$.

Сам извршни SCL код се може реализовати на следећи начин:

```

1 IF NOT #enable THEN
2     #e_out := 0.0;
3     #u_p_out := 0.0;
4     #u_integral_stat := 0.0;
5     #u_i_out := 0.0;
6     #u_out := 0.0;
7 ELSE
8     IF #open_loop THEN
9         // Otvorena petlja
10        #e_out := 0.0;
11        #u_p_out := 0.0;
12        #u_integral_stat := 0.0;
13        #u_i_out := 0.0;
14        #u_out := #w_setpoint;
15
16        ELSIF NOT #pi_mode THEN
17            // P rezim
18            #Kc_P_out := #Kc_in;
19            #e_out := #w_setpoint - #y_actual;
20            #u_p_out := #Kc_P_out * #e_out;
21            #u_integral_stat := 0.0;
22            #u_i_out := 0.0;
23            #u_out := #u_p_out;
24
25        ELSE
26            // PI rezim
27            #Kc_PI_out := #Kc_in;
28            #Ki_step_out := #Ki_in * 0.04; // Ts = 0.04 s
29
30            #e_out := #w_setpoint - #y_actual;
31            #u_p_out := #Kc_PI_out * #e_out;
32            #u_integral_stat := #u_integral_stat + (#Ki_step_out * #e_out);
33
34            // Anti-Reset Windup (ARW)
35            IF #u_integral_stat > 125.0 THEN
36                #u_integral_stat := 125.0;
37            ELSIF #u_integral_stat < 0.0 THEN
38                #u_integral_stat := 0.0;
39            END_IF;
40
41            #u_i_out := #u_integral_stat;
42            #u_out := #u_p_out + #u_i_out;
43        END_IF;
44
45        // Saturacija izlaza regulatora (0 - 125%)
46        IF #u_out > 125.0 THEN
47            #u_out := 125.0;
48        ELSIF #u_out < 0.0 THEN
49            #u_out := 0.0;
50        END_IF;
51    END_IF;

```

Након извршеног писања кода, програмски блок обавезно треба компајлирати и проверити да ли се јављају евентуалне грешке. Ако се након компајлирања блока не јаве грешке можемо прећи на следећи програмски блок.

3.2. Креирање и програмирање симулатора објекта

Након успешног дефинисања и програмирања контролерског блока, следећи корак је израда засебног функцијског блока Simulator објекта, или у овом пројекту FB_Plant (FB2). Овај блок садржи дискретни математички модел објекта управљања (електромотора), чиме се симулира његов динамички одзив у реалном времену на основу управљачког сигнала добијеног из контролерског блока.

Након убацивања новог функцијског блока FB, Поново је потребно дефинисати интерфејс блока, на следећи начин:

FB_Plant									
	Name	Data type	Default value	Retain	Accessible f...	Writa...	Visible in ...	Setpoint	Cor
1	Input								
2	enable	Bool	false	Non-ret...	☒	☒	☒	☐	
3	u_in	Real	0.0	Non-retain	☒	☒	☒	☐	
4	Output								
5	y_out	Real	0.0	Non-retain	☒	☒	☒	☐	
6	InOut				☐	☐	☐	☐	
7	<Add new>				☐	☐	☐	☐	
8	Static				☐	☐	☐	☐	
9	u_stat	Real	0.0	Non-retain	☒	☒	☒	☐	
10	y_stat	Real	0.0	Non-retain	☒	☒	☒	☐	
11	Temp				☐	☐	☐	☐	

Улазни параметри (под групом Input):

- enable (Bool): Дозвола рада симулатора (повезано са тагом mod_motora).
- u_in (Real): Управљачки сигнал добијен из регулатора (matematicko_upravljanje).

Изразни параметри (под групом Output):

- y_out (Real): Тренутни израчунати излаз/одзив објекта (izlazna_vrednost_objekta).

Статичке променљиве (под групом Static):

- y_stat (Real): Статичка променљива која чува вредност излаза из претходног корака одабирања ($y[k-1]$).
- u_stat (Real): Статичка променљива која чува вредност управљачке променљиве из претходног корака одабирања ($u[k-1]$).

Алгоритам унутар FB_Plant блока извршава следећа израчунавања:

1. Провера дозволе рада (enable): Уколико је enable = FALSE, излазна вредност y_out и претходно стање y_stat одмах се ресетују на 0.0.
2. Дискретни прорачун: Када је рад омогућен, извршава се дискретна диференцијална једначина за период одабирања $T_s = 40$ [ms] и временску константу $\tau = 0.3$ [s]:

$$y[k] = 0.8751733 * y[k - 1] + 0.0998613 * u[k - 1]$$

Након израчунавања y_stat , излаз се поставља као $y_out := y_stat$, чиме се чува претходно стање за следећи такт одабирања (40 [ms]). Затим се изједначавају $u_stat := u_in$, односно памти се управљање из текућег такта, које се у следећем такту користи као $u[k-1]$.

Сам извршни SCL код се може реализовати на следећи начин:

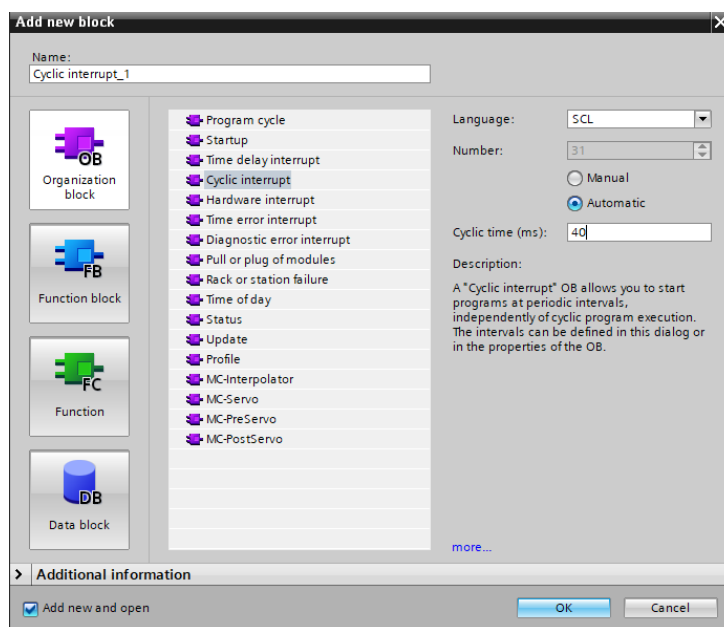
```

1 IF NOT #enable THEN
2     #y_stat := 0.0;
3     #y_out := 0.0;
4     #u_stat := 0.0;
5 ELSE
6     // Diskretni model objekta prvog reda: y(k) = 0.87517 * y(k-1) + 0.09986 * u(k-1)
7     #y_stat := (0.87517 * #y_stat) + (0.09986 * #u_stat);
8     #y_out := #y_stat;
9     #u_stat := #u_in;
10 END_IF;
```

Након извршеног писања кода, програмски блок обавезно треба компајлирати и проверити да ли се јављају евентуалне грешке. Ако се након компајлирања блока не јаве грешке можемо прећи на следећи програмски блок.

3.3. Инстанцирање и повезивање у OB30 (Cyclic interrupt)

Да би се дискретни алгоритми управљања и симулације извршавали у тачно дефинисаним временским интервалима, њихово позивање се не врши у основном циклусу програма (OB1), већ унутар организационог блока са цикличним прекидом који додајемо у задатак OB30 (Cyclic interrupt).



Пре писања кода, при убацивању OB30 блока подешен је параметар одабирања Cyclic time који је постављен на 40 [ms]. Овим се гарантује да PLC прекида извршавање главног програма на сваких 40 [ms] и позива OB30, чиме су физичке временске константе $T_s = 0.04$ [s] употребљене у SCL коду, и у потпуности усклађене са реалним временом извршавања.

Унутар OB30 блока креиране су две засебне инстанце функцијских блокова са својим припадајућим позивним блоковима (Data Blocks):

- Kontrolerski blok_DB (DB1): Инстанца за регулаторски блок.
- FB_Plant_DB (DB2): Инстанца за симулатор објекта.

Повезивањем улаза и излаза ових блокова са глобалним таговима из Default tag table-а формирана је комплетна дигитална управљачка петља са повратном спрегом.

Формирање Data blocks се најједноставније ради превлачењем самих блокова који се позивају из самог Project tree-а директно у место за SCL код OB30 блока.

Редослед извршавања: у OB30 се прво позива FB1, а затим FB_Plant, па регулатор користи излаз објекта из претходног циклуса, $y[k-1]$.

Након убацивања блокова, и формирања одговарајућих дата блокова, може се прећи на следећи корак. Крајњи ОВ30 програмски блок треба изгледати на следећи начин:

```

1  □ "Kontrolerski blok_DB" (enable:="mod_motora",
2      open_loop:="izbor_zadatka",
3      pi_mode:="izbor_rezima_regulatora",
4      w_setpoint:="referentna_vrednost",
5      y_actual:="izlazna_vrednost_objekta",
6      Kc_in:="Kc_HMI_input",
7      Ki_in:="Ki_HMI_input",
8      u_out=>"matematicko_upravljanje",
9      e_out=>"greska_pracenja",
10     u_p_out=>"proporcionalno_dejstvo",
11     u_i_out=>"vrednost_integrala",
12     Kc_P_out=>"Kc_P",
13     Kc_PI_out=>"Kc_PI",
14     Ki_step_out=>"Ki_step");
15 □ "FB_Plant_DB" (enable := "mod_motora",
16     u_in := "matematicko_upravljanje",
17     y_out => "izlazna_vrednost_objekta");
18

```

3.4. Надзорна логика у OB1 (Main)

Иако се основна контролна петља заснована на прекидима извршава унутар OB30, главни организациони блок OB1 (Main) задржава надзорну улогу над целим системом (такозвани супервизор). OB1 се извршава континуирано у оквиру слободног циклуса PLC-а (Cyclic program execution) и користи се за обезбеђивање безбедносних стања и ресетовање свих процесних променљивих када систем није активан.

Унутар OB1 блока имплементирана је логика која проверава статус главног прекидача `mod_motora`, провера искљученог стања (`mod_motora = FALSE`), односно уколико је мотор искључен, надзорна логика форсира ресетовање свих глобалних процесних тагова на нулу. Тиме се спречава задржавање старих вредности на HMI графику, чувању неважећих интегралних сума или слању било каквог управљачког сигнала ка излазима.

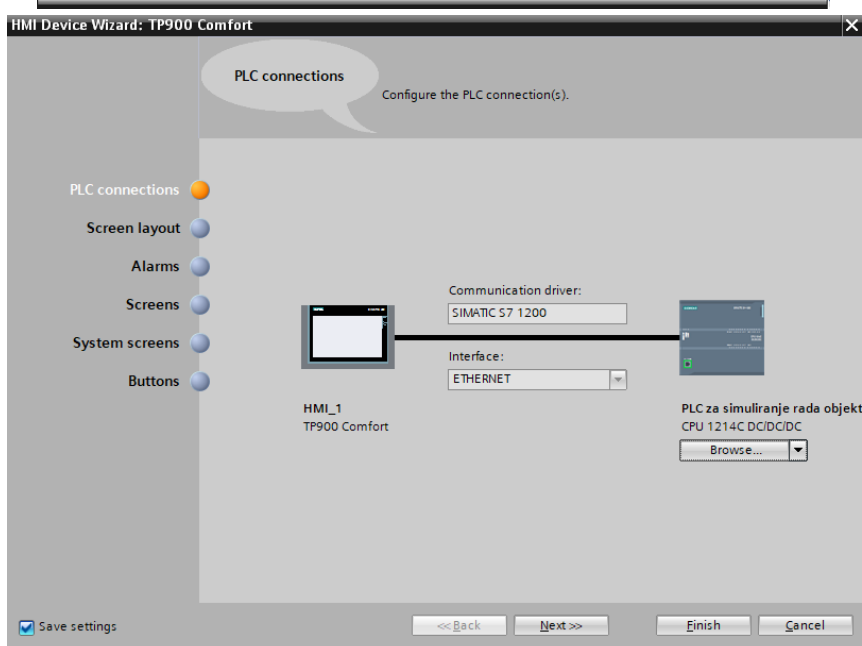
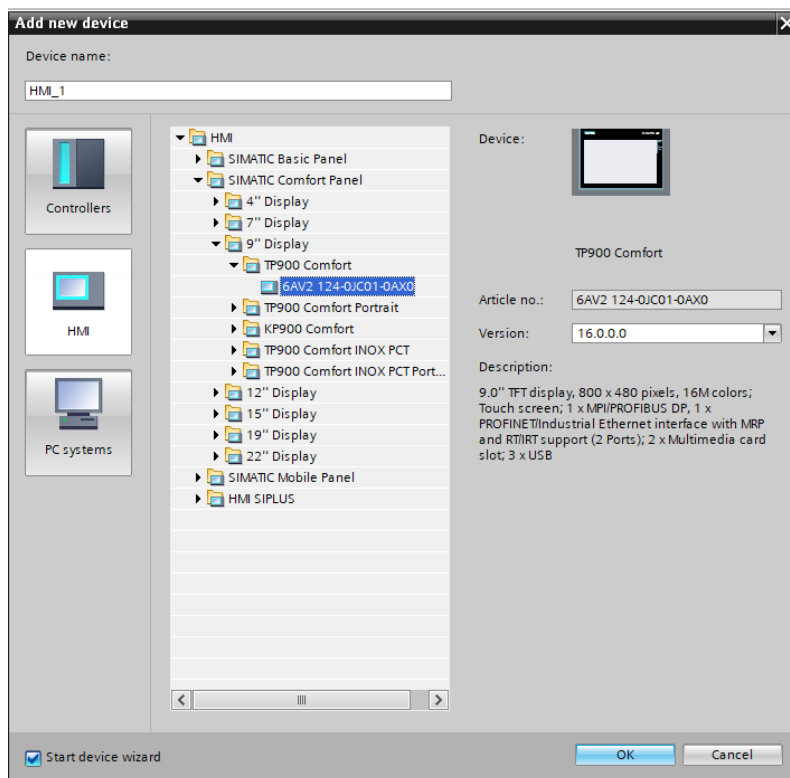
Провера активног стања (`mod_motora = TRUE`). Када је мотор укључен, OB1 препушта потпуну контролу над ажурирањем променљивих OB30 блоку и не врши никакве измене на процесним променљивама.

```
▼ Network 1: .....
Comment
1 // Supervisor logika
2 IF NOT "mod_motora" THEN
3     "izlazna_vrednost_objekta" := 0.0;
4     "matematicko_upravljanje" := 0.0;
5     "greska_pracenja" := 0.0;
6     "vrednost_integrالا" := 0.0;
7     "proporcionalno_dejstvo" := 0.0;
8 END_IF;
```

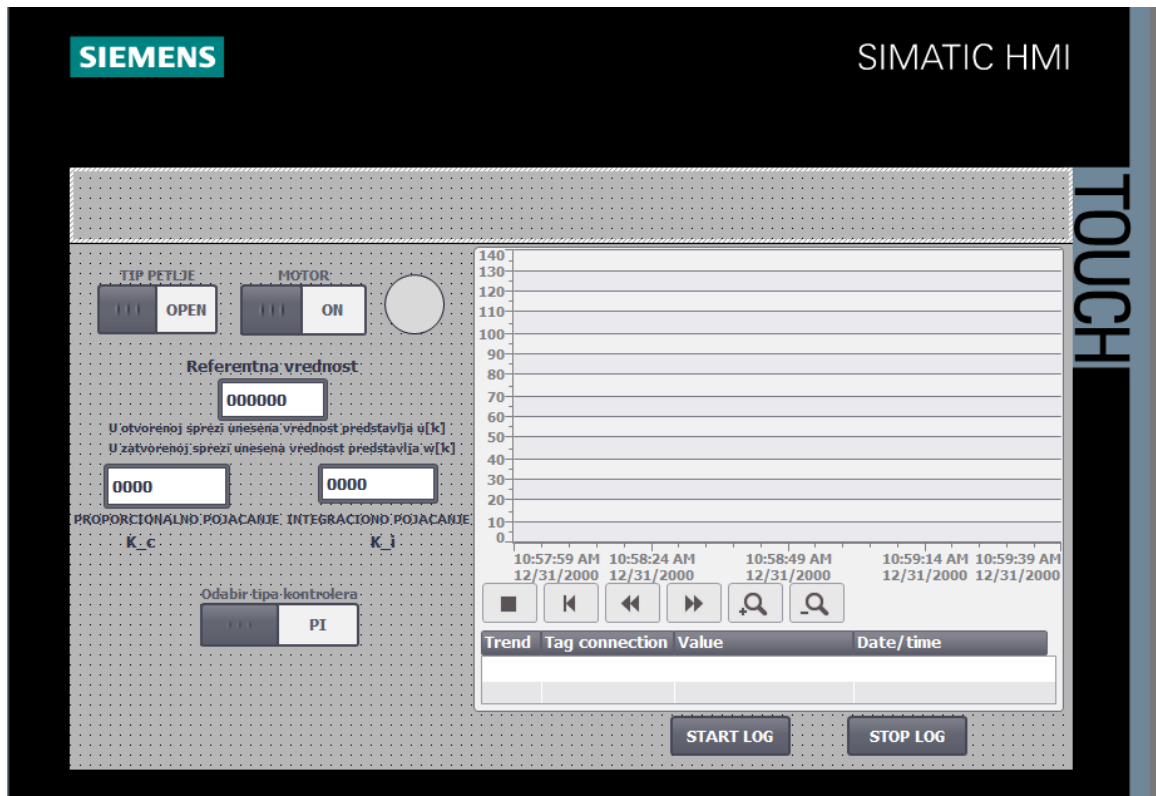
Након комплетирања кода, потребно је поново целокупно компајлирати све промене у PLC-у, и затим након евентуалних преправљања грешака приступити изради следећег корака који јесте убацивање и едитовање HMI панела за читавање вредности.

4. КРЕИРАЊЕ И КОНФИГУРАЦИЈА HMI ЕКРАНА

За потребе визуелизације и директног управљања процесом у реалном времену, потребно је да направимо HMI екран (Root screen) у оквиру WinCC Runtime/Basic HMI окружења. За сврху овог задатка одабран је KTP 900 Comfort PN, чија се имплементација у TIA portal врши одабиром функције Add new device са врха Project tree-ја. Након избора панела, треба да се одабере наш PLC са падајуће листе чиме се остварује конекција између ова два дела хардвера.



Панел који правимо састоји се из више целина:



Главни командни панел се састоји од управљачких елемената:

- Дугме за START / STOP (mod_motora), дозвољава или блокира рад целе симулације и свих управљачких блокова, као и лампица која мења боју у зависности од тога у ком је режиму мотор.
- Прекидач P / PI (izbor_rezima_regulatora), омогућава кориснику тренутну промену алгорита регулатора са пропорционалног на пропорционално-интегрални.
- Прекидач Zatvorena / Otvorena petlja (izbor_zadatka), служи за избор између управљања у затвореној повратној спрези и рада у отвореној петљи.

Улазна поља за параметризацију (I/O Fields):

- Поље за унос референтне вредности (код отворене петље $u[k]$, код затворене петље $w[k]$ или $r[k]$), параметарско улазно поље за задавање жељене вредности излаза.
- Пропорционално појачање (K_c HMI_input), односно поље за унос вредности пропорционалног појачања K_c .
- Интеграционо појачање (K_i HMI_input), поље за унос вредности интегралног појачања K_i .

Централни део екрана заузима Trend View објекат који графички у реалном временом приказује временски одзив система ($y(t)$) у односу на референтну вредност. Trend View служи искључиво за оператерски приказ.

Да би смо себи олакшали унос променљивих, додајемо у Default tag table HMI-а следеће тагове:

Default tag table						
Name	Data type	Connection	PLC name	PLC tag	Address	
izbor_rezima_regulatora	Bool	HMI_Connection...	PLC za simuliranje rada objekta	izbor_rezima_regulatora		
izbor_zadatka	Bool	HMI_Connection...	PLC za simuliranje rada objekta	izbor_zadatka		
izlazna_vrednost_objekta	Real	HMI_Connection...	PLC za simuliranje rada objekta	izlazna_vrednost_objekta		
Kc_HMI_input	Real	HMI_Connection...	PLC za simuliranje rada objekta	Kc_HMI_input		
Kc_P	Real	HMI_Connection...	PLC za simuliranje rada objekta	Kc_P		
Ki_HMI_input	Real	HMI_Connection...	PLC za simuliranje rada objekta	Ki_HMI_input		
mod_motora	Bool	HMI_Connection...	PLC za simuliranje rada objekta	mod_motora		
referentna_vrednost	Real	HMI_Connection...	PLC za simuliranje rada objekta	referentna_vrednost		
<Add new>						

Даље, како би могли да резултате симулације експортујемо ван TIA Portal-а, потребно је да извршимо logging променљивих, које се задају у делу пројектног стабла названог Historical data, на следећи начин:

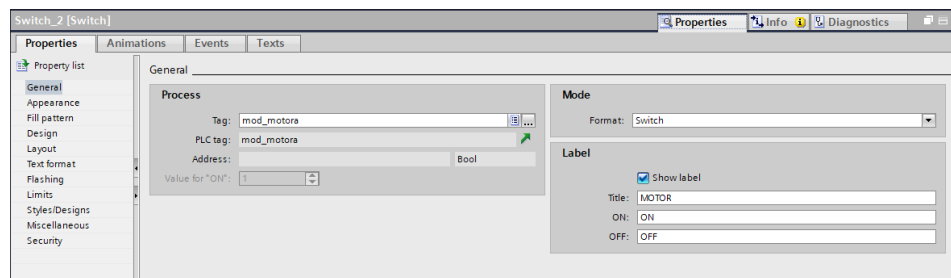
Data logs							
Name	Storage location	Data records	Path	Logging method	Number of sequen...	Fill level	Enable logging at runtime st...
Logging_file	TXT file (Unicode)	1000	IStorage Card SDLogs	Circular log	10	90	☐
<Add new>							

Logging tags							
Name	Process tag	Acquisition mode	Logging cycle	High limit	Low limit	Range for logging limits	Con
izlaz_objekta	izlazna_vrednost_objekta	On change				Within deadband	
Referentna_vrednost	referentna_vrednost	On change				Within deadband	
<Add new>							

За потребе овог задатка вршиће се праћење само две променљиве, излазне вредности објекта, и вредност референтне вредности коју том систему задајемо. Важно је променити Acquisition mode да буде на On change, да би се вредности уписивале у текст фајл након сваке промене која им се деси. Ово логовање служи само за оператерски увид, режим On change не даје једнолико одабране узорке, па се не користи за верификацију дискретног алгорита.

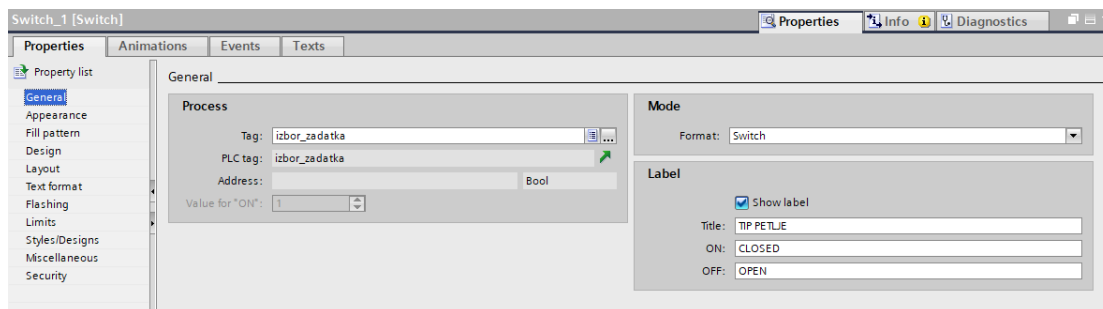
Едитовање HMI панела се врши на делу Project tree-ја под називом Screens, па по дифолту Root Screen, или било који други скрин који додамо касније ако нам је потребан.

Ако кренемо од СТАРТ/СТОП прекидача, његова имплементација је прилично једноставна, из Toolbox-а са десне стране превучемо из секције Elements, прекидач односно Switch на панел. Кликом на тај прекидач и селектовањем Properties са доњег менија можемо едитовати понашање прекидача. Прекидач повезујемо са одговарајућим тагом који контролише дејство паљења односно гашења мотора.

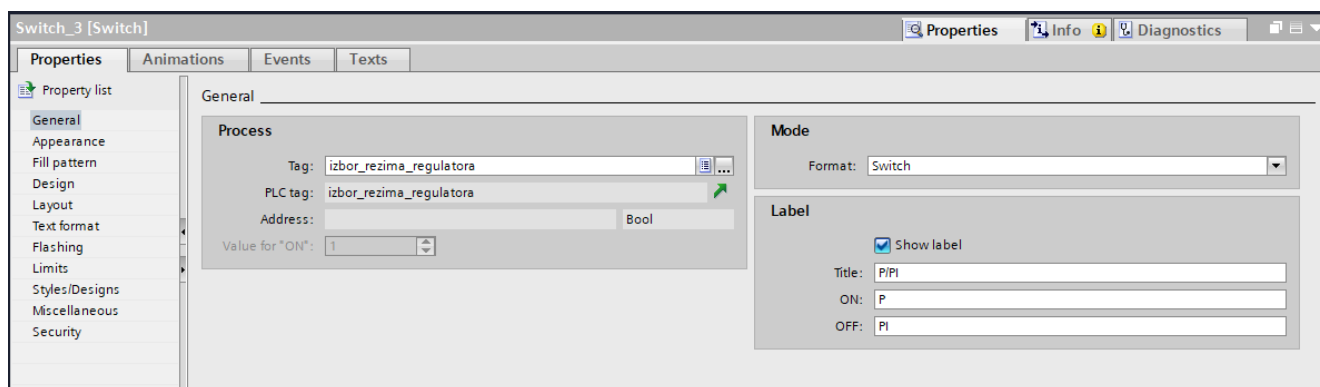


Испитни задатак из Аутоматског управљања

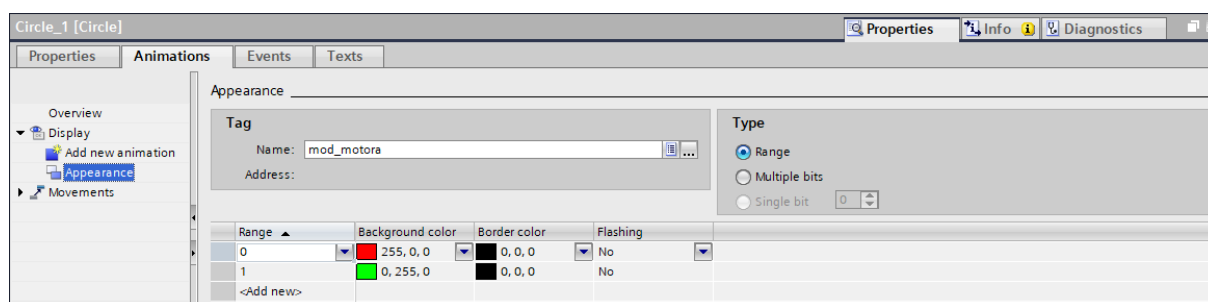
За прекидач који контролише пребацивање задатка из симулације објекта у отвореној петљи и задатка за симулацију затворене повратне спреге, користи се исти принцип као и за СТАРТ/СТОП прекидач, само се мења таг који се везује за функцију дугмета.



За прекидач који дефинише да ли је активан П или ПИ регулатор такође се користи иста логика, при чему се овај прекидач у потпуности искључује при пребацивању симулације у отворену спрегу, јер тада не постоји дејство регулатора, било то П или ПИ.

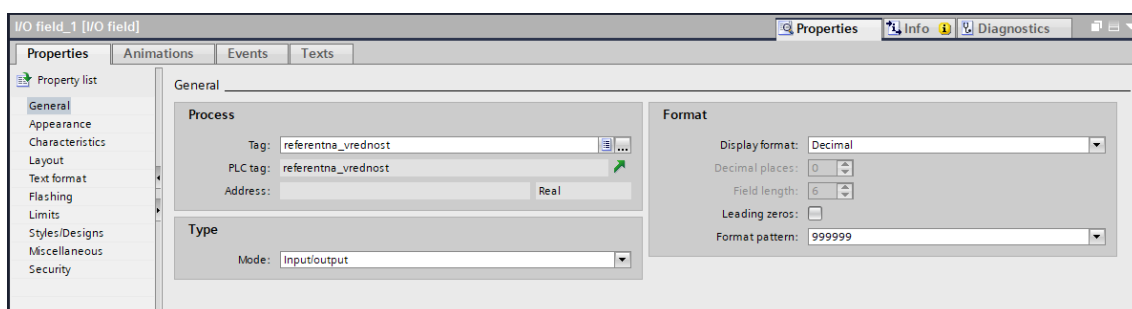


Како би убацили лампицу која мења боју у зависности од статуса мотора, и Toolbox-а на панел превлачимо из дела Basic objects – Circle. Пошто је статус мотора једноставна бинарна промена јединица и нула, у делу Animations од селектованог круга можемо изабрати Display -> Appearance и повезати понашање сијалице и статуса мотора поновно бирајући таг mod_motora, а затим испод тога дефинисати боје сијалице за одговарајуће модове мотора. У мом случају зелено за рад мотора, а црвено док је искључен.

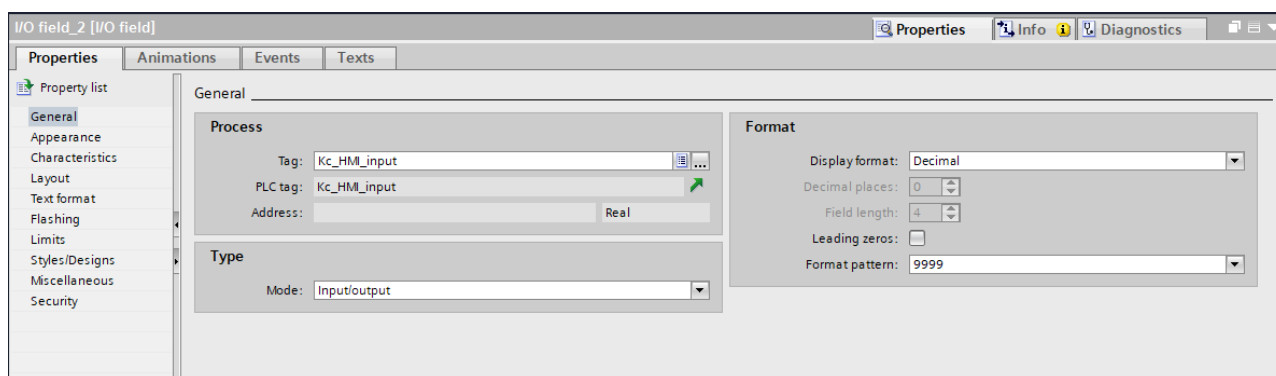


За остала поља у које можемо да унеси вредности параметара управљања која су нам потребна, из Toolbox-а са десне стране из одељка Elements бирамо I/O Field, и превлачимо га на наш панел. Затим у менију Properties ово поље повезујемо са одговарајућим тагом и дефинишемо децимални формат унешеног броја за свако поље које додајемо, односно:

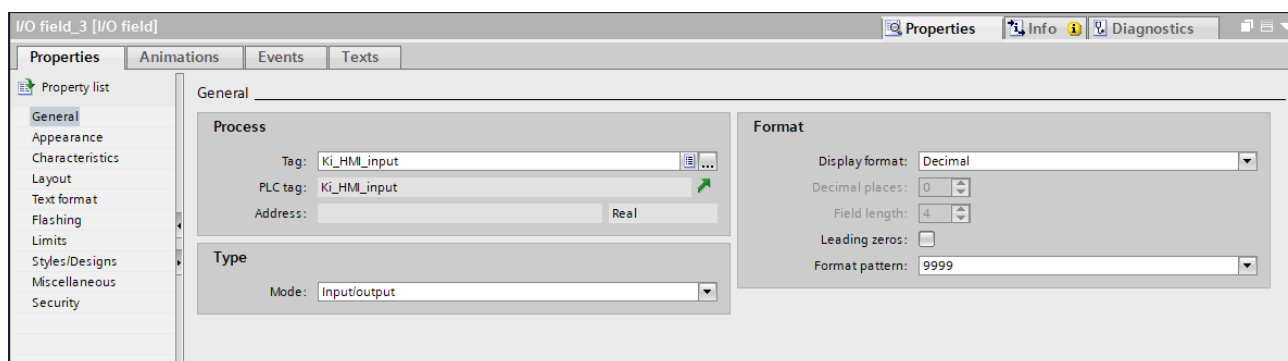
За референтну вредност:



За вредност пропорционалног појачања:

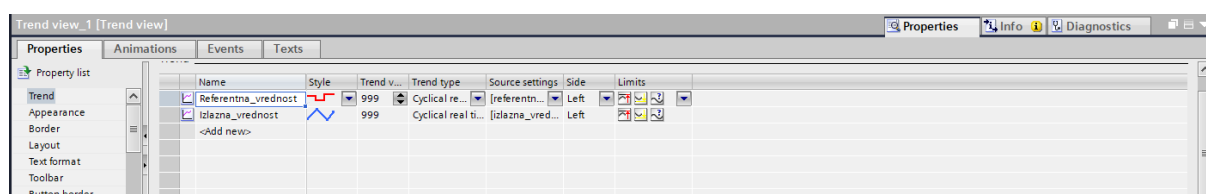


Односно, за вредност интеграционог појачања регулатора:



За убацивање самог графика користимо из Toolbox-а елемент назван Trend View (или f(x) Trend View ако је коришћен Comfort панел при креирању HMI-а), на следећи начин:

Потребно је подесити Trend type да буде Cyclical real time за оператерски приказ, освежавање ових вредности није синхронизовано са OB30, па не гарантује приказ сваког узорака од 40 [ms]). Убацујемо две линије, једна која представља референтни сигнал r[k] (tag referentna_vrednost), а друга, у овом случају плава представља реалну брзину коју за тај сигнал постиже мотор, изражену у процентима (tag izlazna_vrednost_objekta).

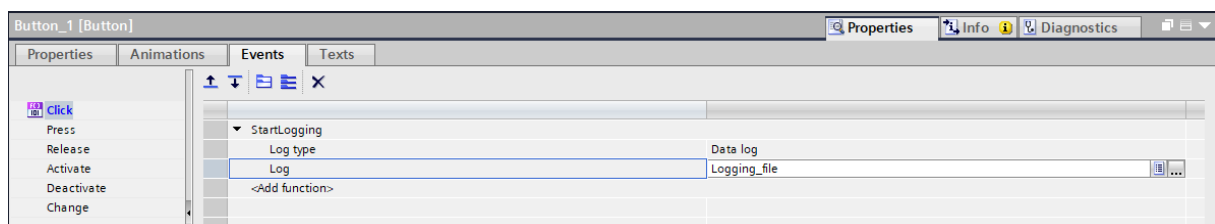


За потребе додатне анализе одзива система и верификације управљачког алгоритма, на HMI екрану су имплементирају командни дугмићи за управљање евиденцијом података (Data Logging). Ови дугмићи омогућавају оператеру да ручно започне или заустави снимање процесних променљивих у реалном времену.

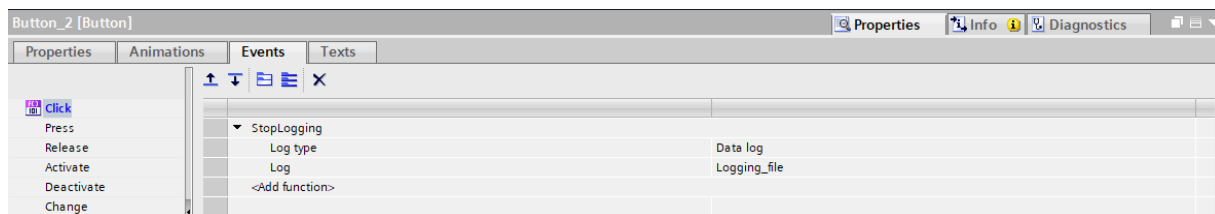
Активирањем ове функције, подаци се аутоматски записују у текстуалну датотеку у меморији контролера, што омогућава њихов каснији извоз и графичку обраду у спољним софтверским алатима (као што је MATLAB у нашем случају) ради упоређивања реалног одзива са симулационим моделима. Овако снимљен лог је само илустрација; за квантитативну верификацију користи се искључиво TIA Portal Trace.

Дугмићи се убацују из Toolbox-а као Button, и то са следећим својствима:

За дугме Start log:



Односно за дугме Stop log:



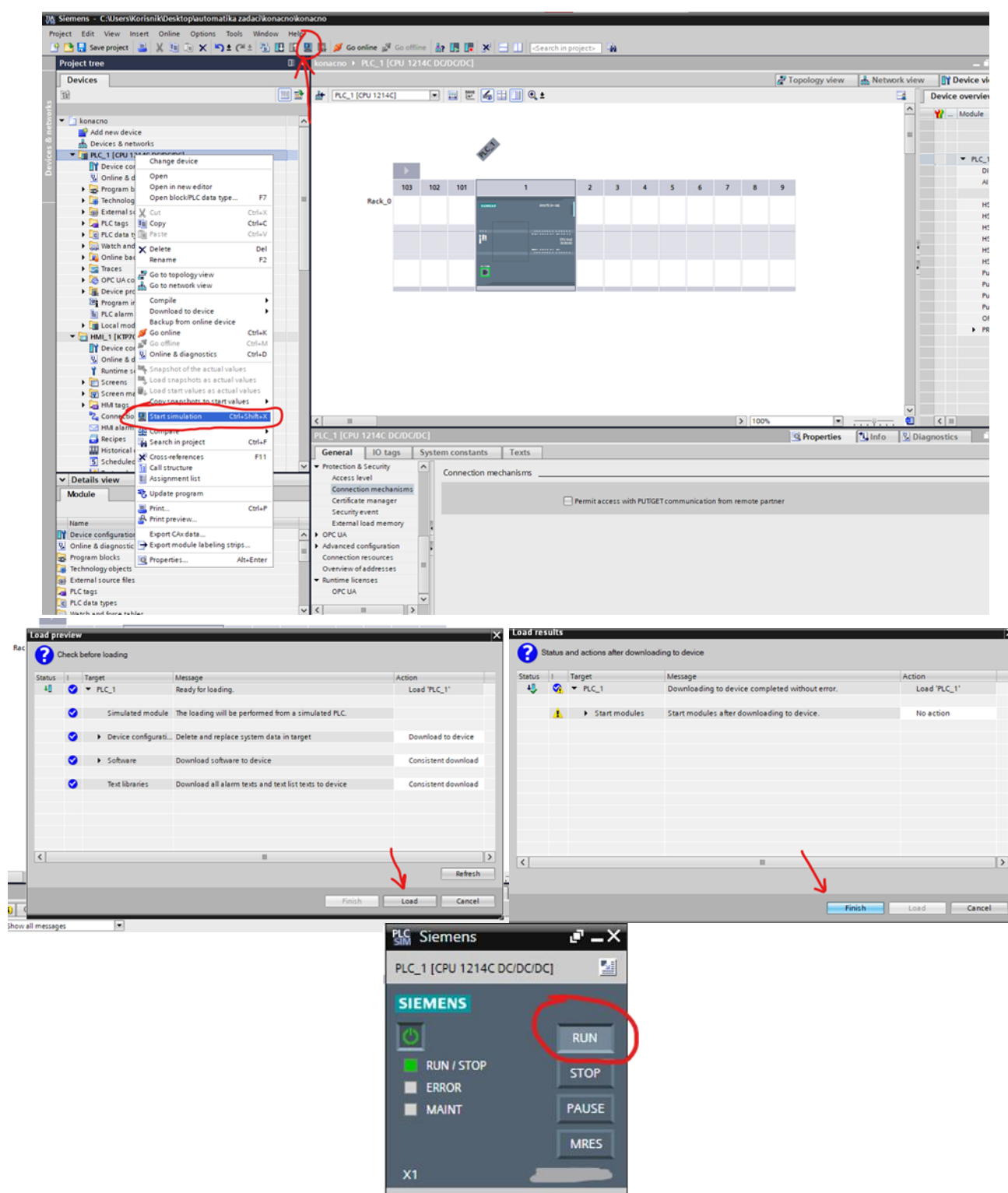
Добијени текстуални фајл ће се по дифолту чувати на адреси Local disk/[Storage Card SD/USB_X60.1]/[име лог фајла].

Када сматрамо да смо завршили едитовању HMI панела, можемо компајловати сав хардвер за сваки случај и, ако нема грешака наставити даље.

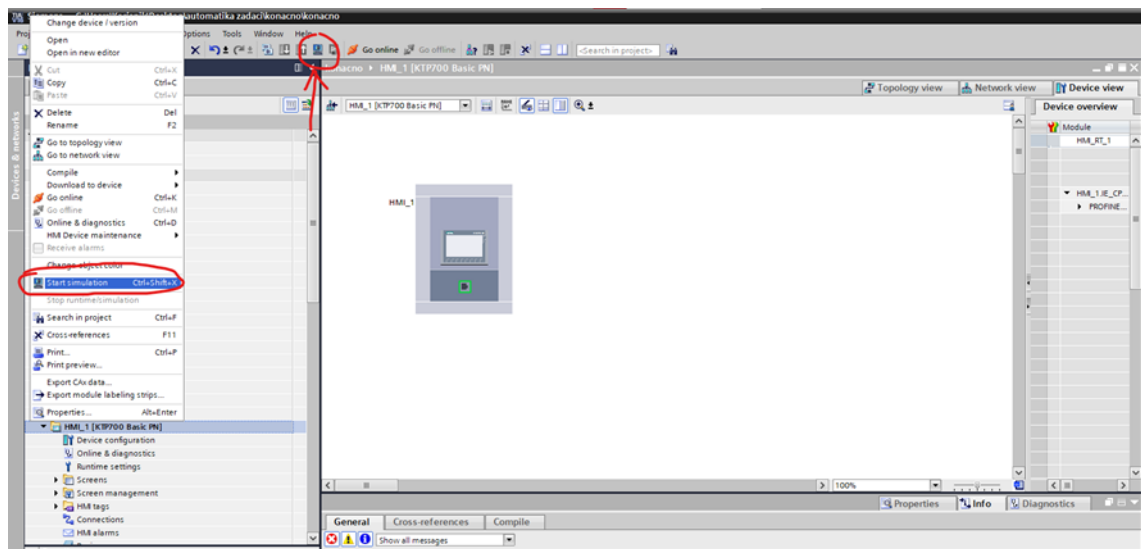
5. ПОКРЕТАЊЕ СИМУЛАЦИЈЕ И ДИСКУТОВАЊЕ РЕЗУЛТАТА

Након завршеног подешавања HMI панела, можемо покренути симулацију и погледати резултате.

Покретање симулације у TIA Portalu се врши у више корака. Прво је потребно да покренемо симулацију самог PLC дела, као на сликама:



Затим покрећемо симулацију HMI-а на исти начин као и PLC, односно:



Када се заврши компајлирање и отвори симулирани HMI, спремни смо за симулације.

Како би смо упоредили управљање постигнуто П и ПИ контролером, спровешћемо више симулација, прво ћемо погледати графике одзива система у отвореној спрези, затим под утицајем П и ПИ контролера респективно. Упоредо са симулацијама које радимо овде, радимо симулације и у MATLAB-у и у Simulink-у, и упоређујемо резултате како би доказали тачност симулације.

Наравно, важно је напоменути да резултати у овом поглављу представљају функционални тест на HMI Trend View-у (односно оператерски приказ). Квантитативна анализа заснива се на TIA Portal Trace подацима, у следећим поглављима.

5.1. Рад система у отвореној петљи

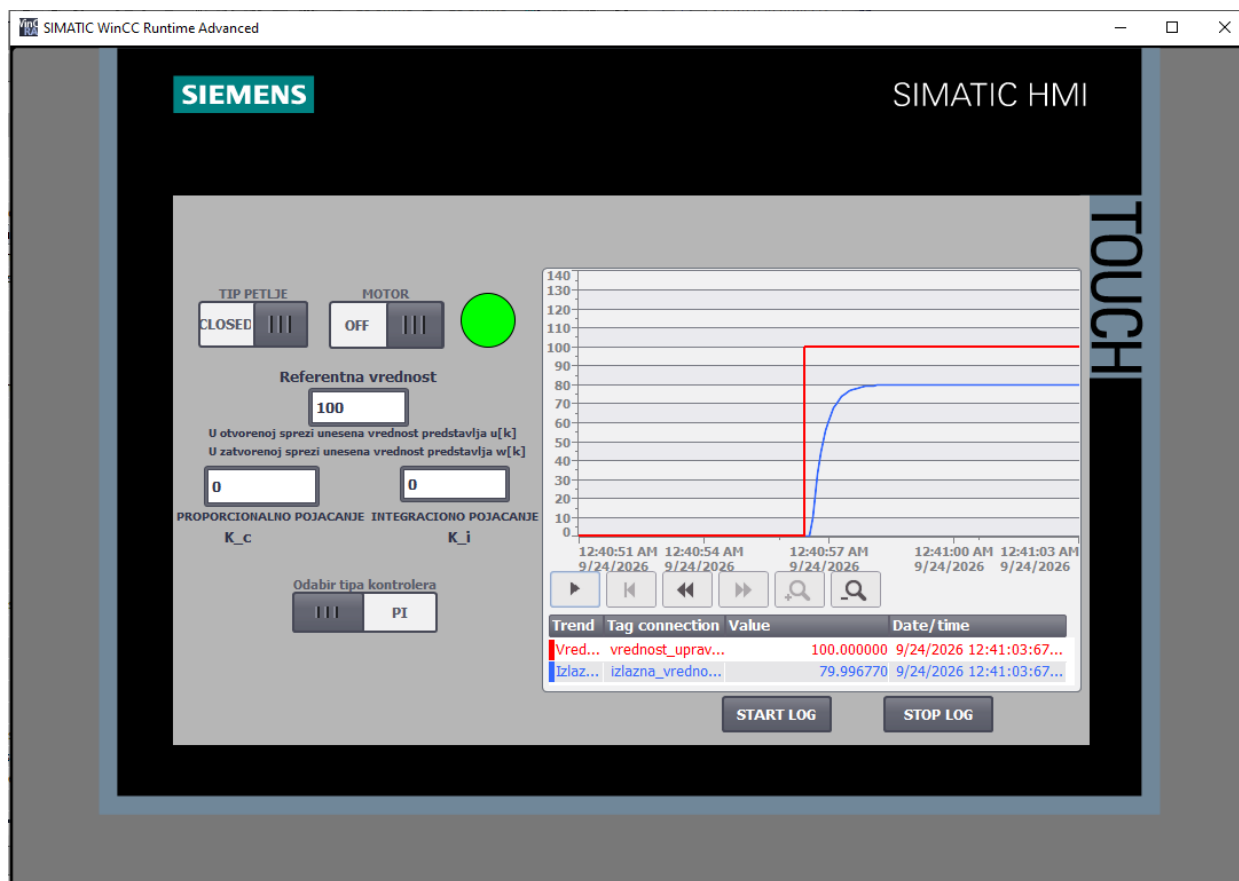
Параметри за следећу симулацију:

- Тип петље: Отворена
- Тип регулатора: Нема активног регулатора
- $u_{OL}[k] \rightarrow G_d(z) \rightarrow y[k]$

Излазни параметри

- $u_{OL}[k] = 100$
- $e[k]$ се у отвореној спрези не рачуна ($e_{out} = 0$)
- $y_{st} = 80$

Како би смо анализирали одскочни сигнал који објекат произведе побудом на степ сигнал вредности 100, потребно је да прво на панели укључимо сам мотор, и затим задамо управљачки сигнал од 100. За овакве параметре, на графику се јавља следећа слика:



Са слике се види да за побудни сигнал од 100, објекат постиже излазну вредност од тек 80 посто, уз доста спор одговор система на побудни сигнал. Ово се у потпуности поклапа са нашим објектом из ЕПФЛ вежби који услед трења и губитка енергије постиже тек вредност од 80 посто од управљачког сигнала.

У овом кораку је такође погодно извршити логирање вредности тако што притиснемо дугме за старт пре укључивања мотора и стоп, након постизања стационарне вредности објекта. Добијени текстуални фајл ће касније бити разматран при анализи резултата.

5.2. Рад система у затвореној повратној спрези

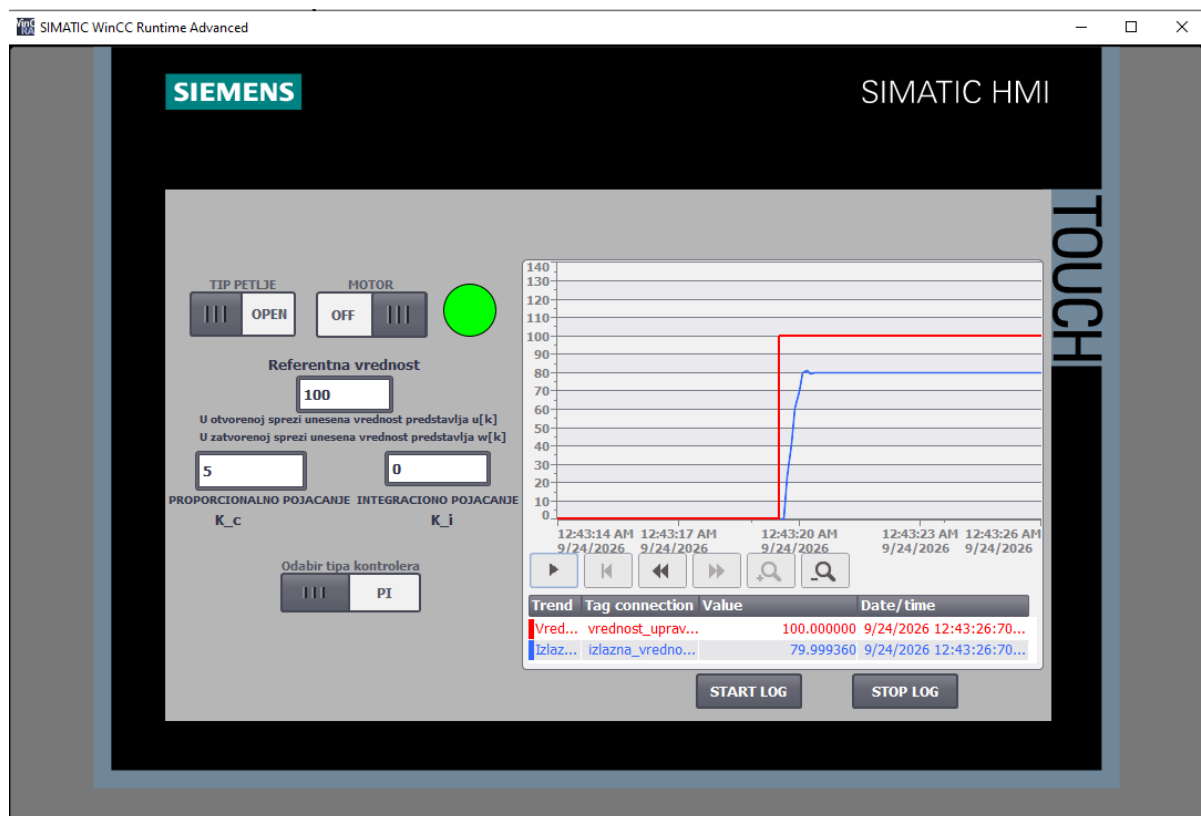
Улазни параметри за следећу симулацију:

- Тип петље: Затворен
- Тип регулатора: П регулатор
- $w[k] = 100$
- $K_c = 5$

Излазни параметри:

- $w[k] = 100$
- $e_{st} = 20$
- $u_{st} = 100$
- $y_{st} = 80$

Ако сада затворимо повратну спрегу притиском на прекидач на панелу, потребно је да унесемо вредности одговарајућих појачања које користе контролери при управљању над објектом. Па тако уношењем пропорционалног појачања од 5, ми за исти побудни сигнал од 100, добијамо следећи график:



Пропорционално појачање 5 нам убрзава одзив система (што се види из тога да је плава линија ближа црвеној него на претходном графику), али и даље постоји грешка стационарног стања. Ако ми сада у овој конфигурацији задатка повећавамо јачину пропорционалног појачања на неке високе бројеве, приметимо да то доводи само до смањивања стабилности система, док се упоредно смањује и грешка стационарног стања, али она никад не постиже жељену вредност од 100, због непоседовања интеграционог члана при рачуну.

Улазни параметри за следећу симулацију:

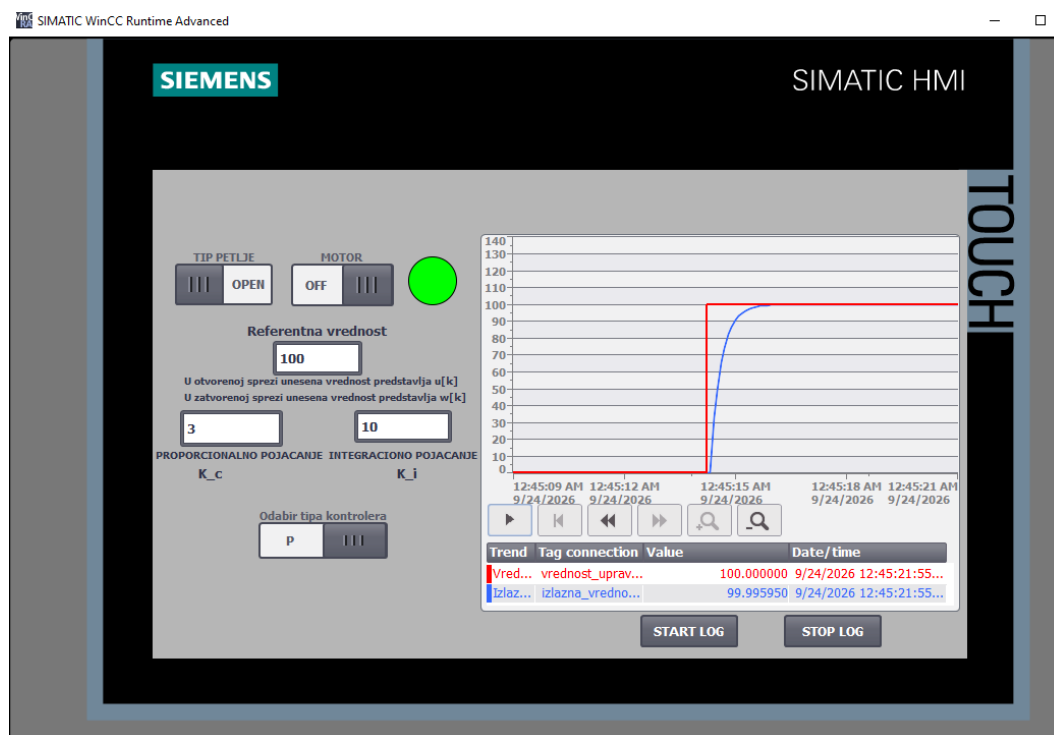
- Тип петље: Затворен
- Тип регулатора: ПИ регулатор
- $w[k] = 100$
- $K_c = 3$
- $K_i = 10 \text{ s}^{-1}$

Изразни параметри:

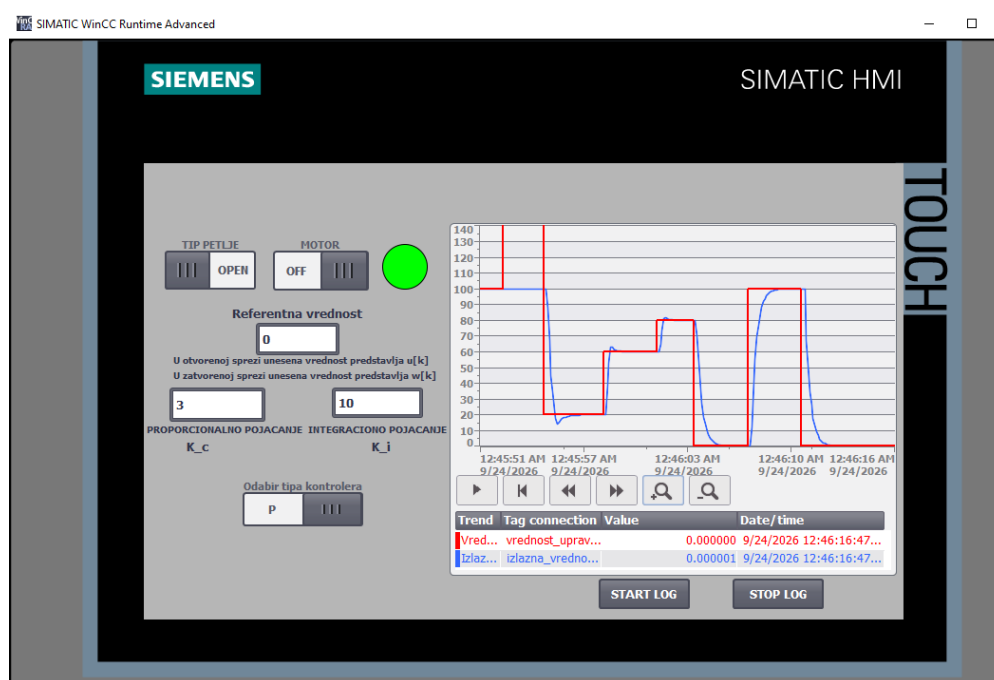
- $w[k] = 100$
- $e_{st} = 0$
- $u_{st} = 125$
- $y_{st} = 100$

Испитни задатак из Аутоматског управљања

Ако пак пребацимо симулацију на задатак у затвореној повратној спрези под регулацијом ПИ контролера, и задамо вредност интеграционог појачања 10, добија се следећи график:



Пројектовани контролер у потпуности успева да елиминише грешку стационарног стања, уз задржавање брзине система које нам обезбеђује пропорционално дејство. Овај регулатор не само што елиминише грешку стационарног стања, већ се и врло добро стабилизује ако га пореметимо разним скоковитим променама вредности управљања, што се види са слике:



Наравно, вредности појачања регулатора се могу мењати произвољно, али за потребе задатка је потребно користити најефективнију комбинацију појачања, што је у овом случају $K_c = 3$ и $K_i = 10$.

6. Снимање и анализа временских одзива система (TIA Portal Trace)

За физичку промену алгоритма управљања и дигиталног модела објекта, кључно је да аквизицију података вршимо са стриктно детерминистичким периодом одабирања.

У ту сврху, унутар PLC програма искоришћен је циклични прекидни блок OB30, који смо направили и попунили у претходним поглављима, а чији је период извршавања конфигурисан на фиксних $T_s = 40$ [ms]. Све кључне дискретне једначине – дигитални модел објекта $G(s) = 0.8/(0.3s + 1)$ и дискретни PI регулатор са Anti-Windup заштитом, извршавају се искључиво унутар овог блока.

За снимање ових мерних сигнала, користимо интегрисани системски алат TIA Portal Trace. Интеграција и подешавање аквизиције спроведени су према следећим параметрима.

Вршимо синхронизовање са OB30 блоком, тако што тригер и “sampling” Trace алата директно повезујемо са цикличним прекидом OB30, чиме је обезбеђено да се сваки узорак снима на сваких 40 [ms].

Вршимо раздвајање од HMI слоја, HMI систем (Trend View и Historical Data) конфигурисан је за оператерски надзор са споријим периодом освежавања (100 [ms]) ради смањења оптерећења комуникације, док се сви подаци за детаљну анализу и графички приказ искључиво преузимају из TIA Portal Trace-а.

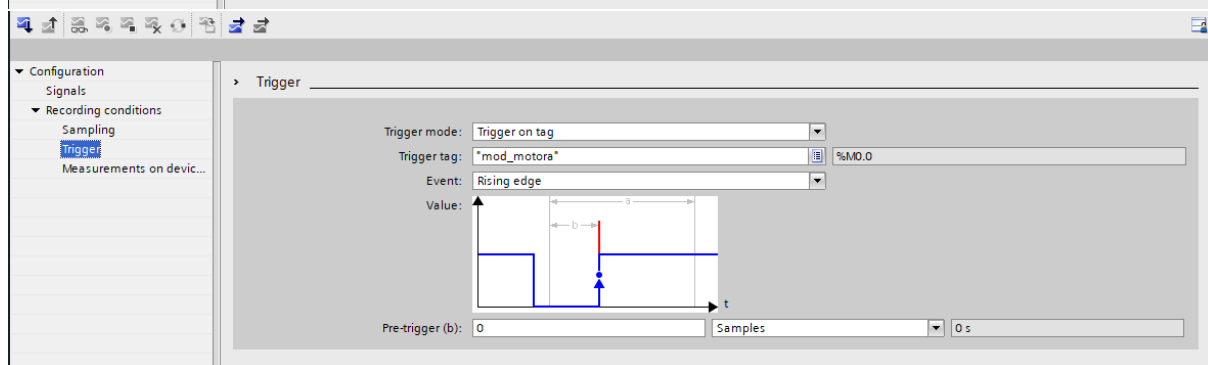
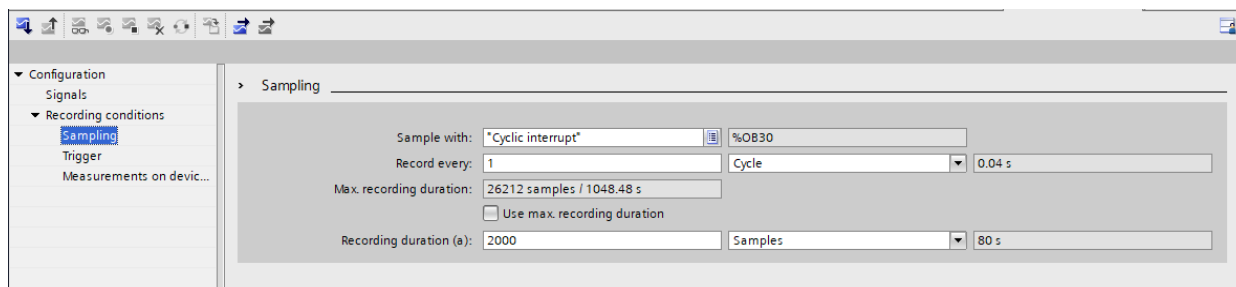
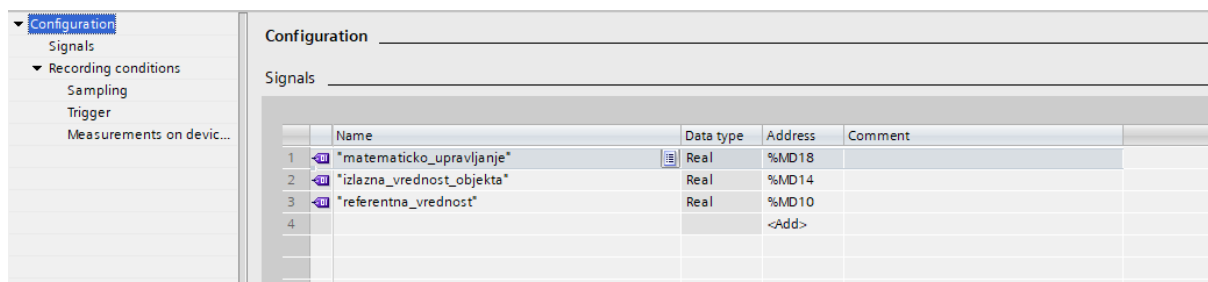
HMI Trend View и Historical Data служе искључиво за оператерски приказ: њихово освежавање није синхронизовано са OB30, а Historical Data у режиму On change не даје једнолико одабране узорке. Квантитативна верификација спроводи се искључиво на подацима TIA Portal Trace-а, одабираним цикличним прекидом OB30 ($T_s = 0.04$ [s]).

Током свих експеримената истовремено су праћена три кључна сигнала: *referentna_vrednost*, *w[k]* (односно референтни СТЕП сигнал), *matematicko_upravljanje u[k]* (излаз регулатора), *izlazna_vrednost_objekta y[k]* (вредност одзива дигиталног модела).

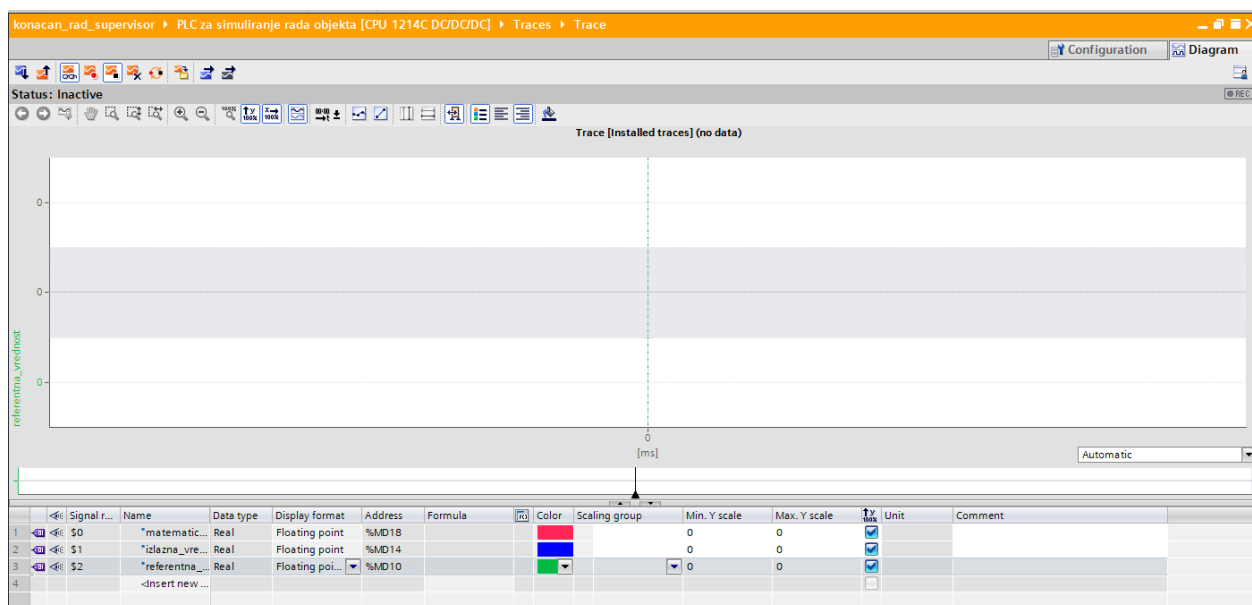
Експорт података вршимо тако што снимљене временске дијаграме извеземо у .csv/.txt форматима, што омогућава њихово директно учитавање и графичку обраду у MATLAB окружењу ради поређења са теоријском симулацијом.

Прво додајемо нови Trace из Project tree-а са леве стране, кликом на Add new trace, са следећим параметрима:

Испитни задатак из Аутоматског управљања



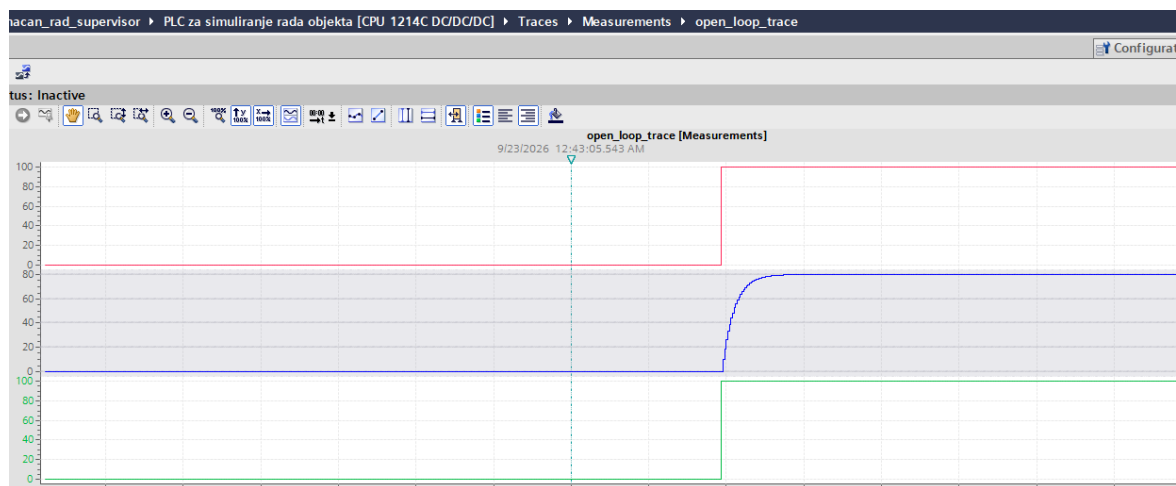
Када кликнемо дугме из горњег менија за трасфер дате конфигурације на наш систем (Transfer trace configuration to device), отвориће се следећи прозор:



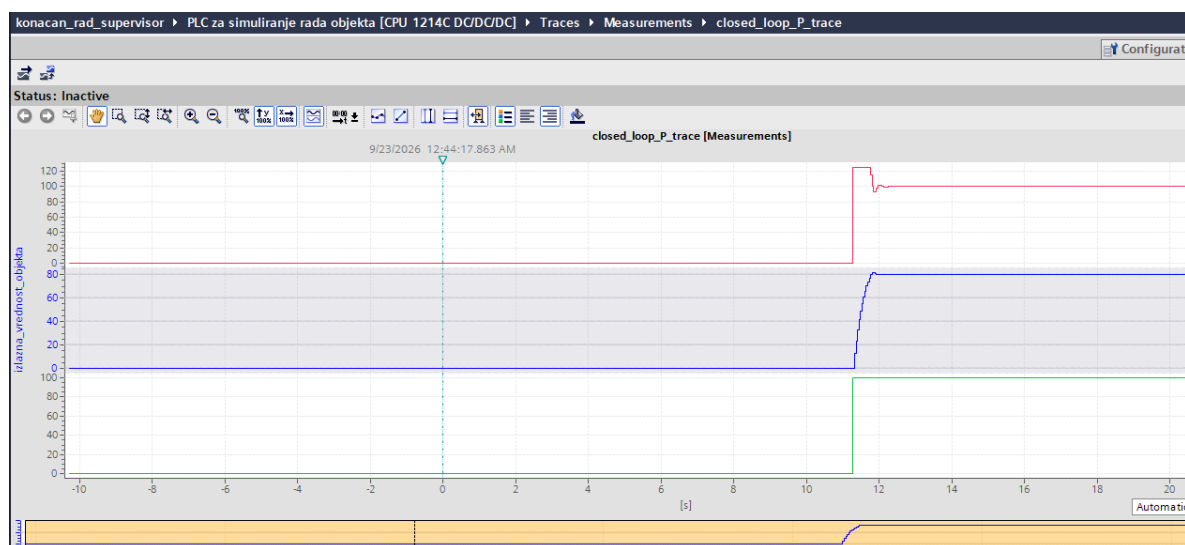
Напомена да уколико сам виртуелни ПЛЦ није активан, нећемо моћи да трансферујемо конфигурацију овако, док не упалимо ПЛЦ. Након овога, кликом на другме за почетак снимања отпочињемо снимање резултата, али тек од оног тренутка када упалимо сам

мотор на HMI панели. Тако, можемо направити „snapshot“-ове наших резултата и експортирати их касније.

У сврху испитивања исправности сваког дела овог пројекта, извршена су 4 мерења, са резултатима као на сликама:

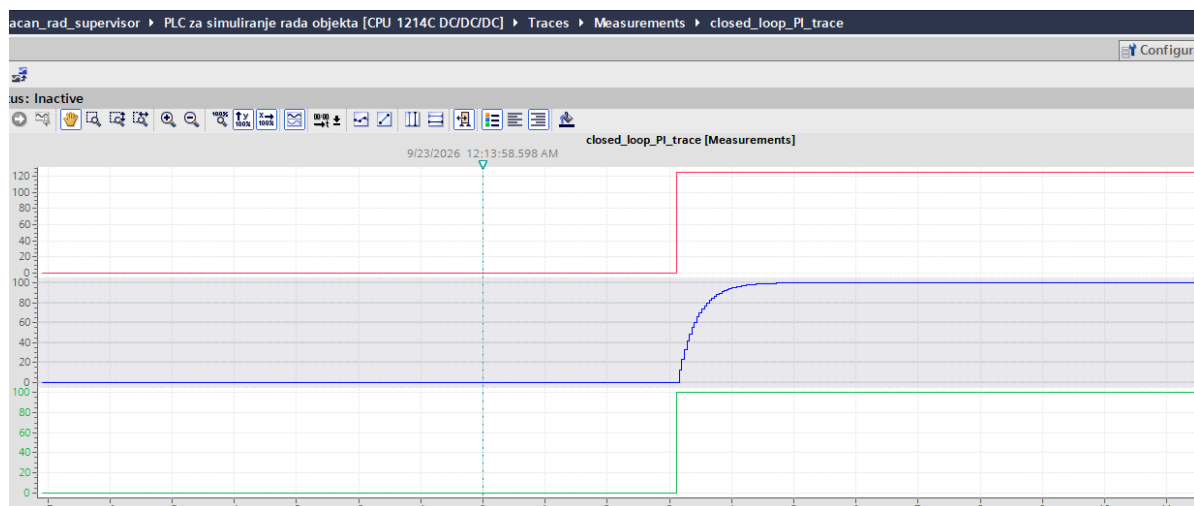


Слика 1: Снимак одзива система у отвореној вези на СТЕП сигнал вредности 100

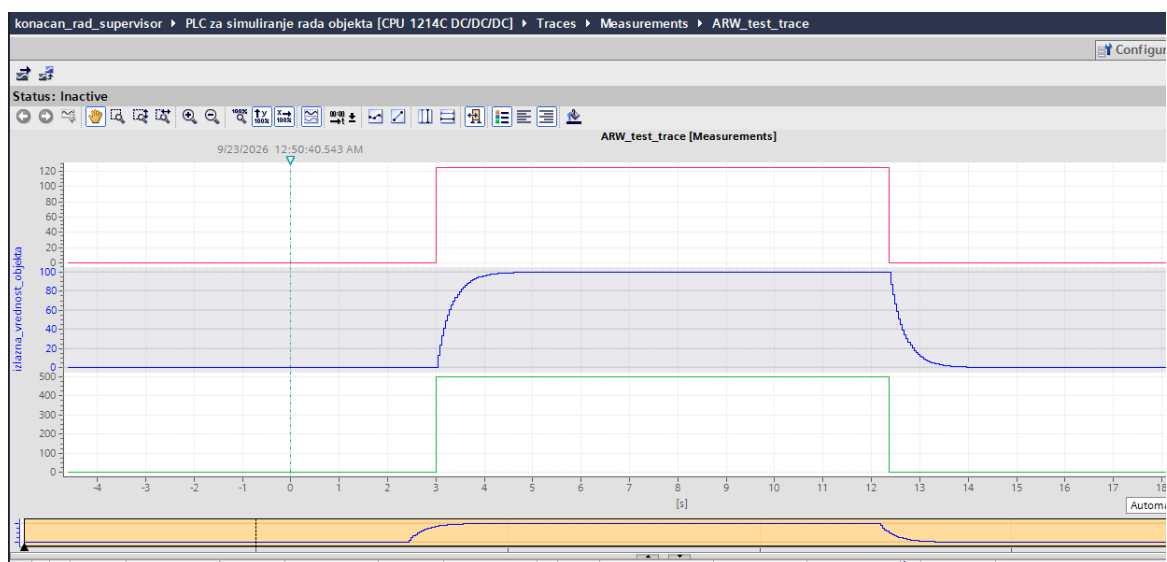


Слика 2: Снимак одзива система у затвореној повратној спрези под П регулатором ($w = 100$)

Испитни задатак из Аутоматског управљања



Слика 3: Снимак одзива система затвореној повратној спрези под ПИ регулатором ($w = 100$)



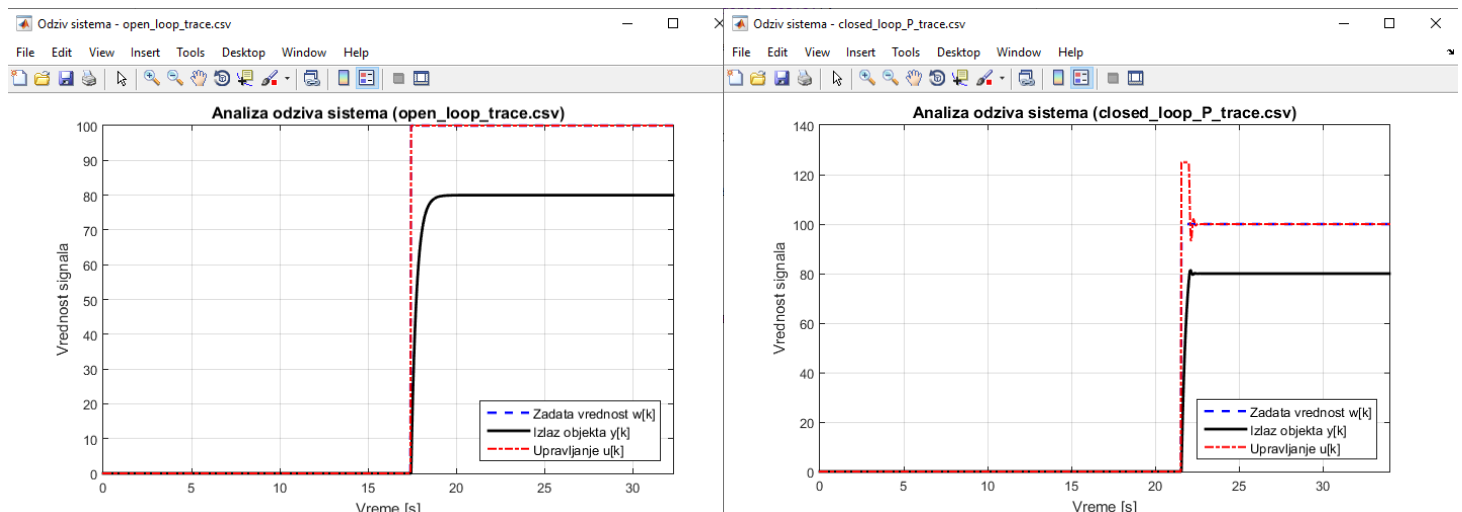
Слика 4: Снимак одзива система при задавању улазне вредности од 500, услед доказивања деловања ARW механизма, односно да не постоји препуњавање интегратора које успорава одзив система

Испитни задатак из Аутоматског управљања

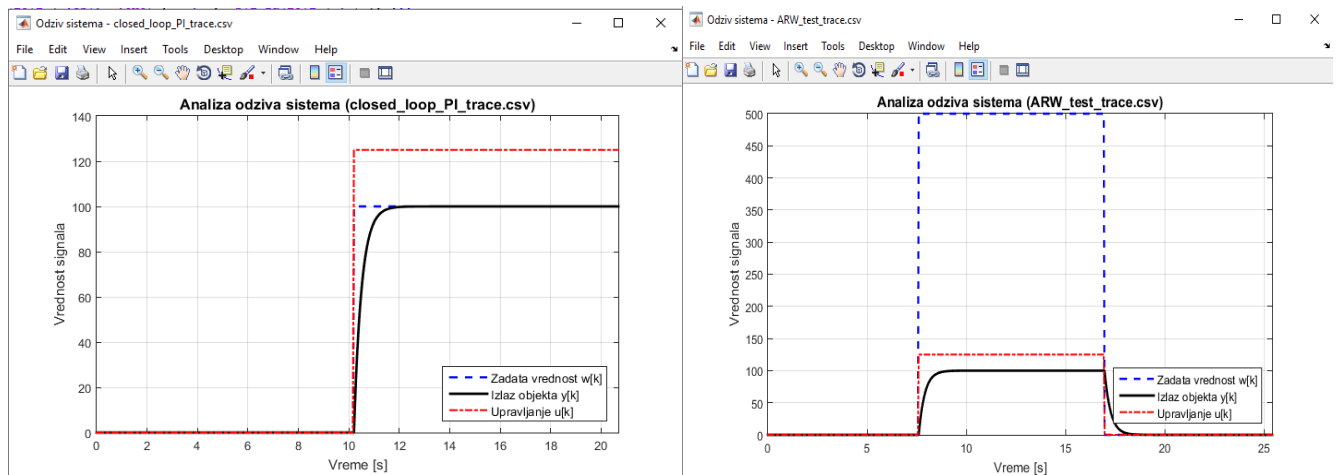
Ако добијене вредности сада екпортујемо ван TIA Portal-а, добићемо 4 .csv фајла, чије исцртавање на график може обавити следећа скрипта:

```
csv_citac.m* x +
1 clear all;
2 clc;
3 close all;
4
5 [filename, pathname] = uigetfile({'*.csv;*.txt', 'Trace fajlovi (*.csv, *.txt)'; '*.*.txt', 'Svi fajlovi (*.*)'}, ...
6 'Odabir fajla');
7
8 if isequal(filename, 0) || isequal(pathname, 0)
9     disp('otkazano');
10    return;
11 end
12
13 fullFilePath = fullfile(pathname, filename);
14
15 try
16     rawData = importdata(fullFilePath);
17     if isstruct(rawData)
18         matrixData = rawData.data;
19     else
20         matrixData = rawData;
21     end
22 catch
23     matrixData = readmatrix(fullFilePath);
24 end
25
26 if isempty(matrixData)
27     error('greska');
28 end
29
30 [rows, cols] = size(matrixData);
31
32 Ts = 0.04;
33 time = (0:rows-1) * Ts;
34
35 u = matrixData(:, end-2);
36 y = matrixData(:, end-1);
37 w = matrixData(:, end);
38
39 figure('Name', ['Odziv sistema - ' filename], 'NumberTitle', 'off', 'Color', [1 1 1]);
40
41 plot(time, w, 'b--', 'LineWidth', 1.8); hold on;
42 plot(time, y, 'k-', 'LineWidth', 2.0);
43 plot(time, u, 'r-.', 'LineWidth', 1.5);
44
45 grid on;
46 title(['Analiza odziva sistema (' filename ')'], 'Interpreter', 'none', 'FontSize', 12);
47 xlabel('Vreme [s]', 'FontSize', 11);
48 ylabel('Vrednost signala', 'FontSize', 11);
49
50 legend(['Zadata vrednost w[k]', 'Izlaz objekta y[k]', 'Upravljanje u[k]'], ...
51 'Location', 'southeast', 'FontSize', 10);
52
53 xlim([0 max(time)]);
```

Покретањем скрипте и бирањем одговарајућих фајлова, добијамо следеће дијаграме:



Испитни задатак из Аутоматског управљања



Сатурација се примењује на излаз регулатора (управљачки сигнал $u[k]$ је у опсегу $[0, 125]$). Сам излаз објекта $y[k]$ нема уграђен блок сатурације, већ његова максимална вредност физички зависи од управљања и појачања објекта $K=0.8$, па при $u_{\max}=125$ излаз природно достиже $y_{\max} = 0.8 \cdot 125 = 100$.

Сада, ако извршимо појединачну анализу ових графика закључујемо следеће:

На дијаграму одзива за отворену спрегу (слика са `open_loop_trace.csv`) приказан је одзив дигиталног модела објекта на директну одскочну побуду управљања амплитуде $u[k] = 100$. Са дијаграма се јасно види да излаз објекта $y[k]$ монотонно расте све док не достигне стационарну вредност од 80. Овакав одзив у потпуности потврђује постављени математички модел објекта првог реда $G(s) = 0.8/(0.3s + 1)$, где статичко појачање износи $K = y_{st}/u_{st} = 80/100 = 0.8$. Временски ток одзива прати теоријску временску константу од $\tau = 0.3$ [s], а одсуство било каквог кашњења или прескока на излазу потврђује да је дискретизација са периодом од $T_s = 40$ [ms] унутар блока ОВ30 спроведена и физички и математички тачно.

На дијаграму затворене петље регулисане П регулатором (`closed_loop_P_trace.csv`) приказан је одзив на скок референце са $r[k] = 0$ на $r[k] = 100$ (скок се дешава око 22 [s] након тригера, који означава почетак снимања). У првом такту је $e = 100$, па би $K_c \cdot e$ износило 500; управљање се зато сатурише на $u_{\max} = 125$ и остаје на граници све док грешка не падне испод 25, након чега се управљање смањује и стабилизује на $u_{st} = 100$. Излаз $y[k]$ достиже $y_{st} = 80$ уз мали прескок до 81.37, који је последица кашњења од једног такта у повратној грани (регулатор чита $y[k-1]$, што је објашњено у претходним главама) и сатурације. Преостала грешка $e_{st} = 20$ карактеристична је за чисту П регулацију: $y_{st} = K_c K / (1 + K_c K) \cdot r = 4/5 \cdot 100 = 80$.

На дијаграму одзива са ПИ регулатором (`closed_loop_PI_trace.csv`) приказано је управљање у затвореној спрези где је додато интегрално дејство ради елиминације стационарне грешке. Приликом одскочне промени задате вредности на $w[k] = 100$, управљачки сигнал $u[k]$ снажно реагује и у почетном тренутку, и достиже вредност од 125. За разлику од П регулатора, континуално дејство интегратора омогућава да се управљање одржи на нивоу од 125, што као директну последицу има довођење излаза објекта $y[k]$ тачно на задану вредност од 100, пошто је $125 \cdot 0.8 = 100$. Излаз објекта показује гładак апериодски прелазни процес без прескока и са нултом стационарном

грешком и одзиву ($e_{st} = 0$), чиме су у потпуности испуњени сви квалитативни захтеви постављени пред наш ПИ регулатор. Пошто је управљање у почетку засићено ($u = 125$), одзив није линеарни ПИ одзив са временском константом затвореног кола $\tau_{cl} = 0.125$ [s], већ одзив засићеног система: 98% задате вредности достиже се за око 1.2 [s] (линеарни модел без сатурације: око 0.5 [s]). Због тога се поређење са MATLAB/Simulink моделом мора вршити моделом са истом сатурацијом и истим кашњењем.

- Реализација ARW механизма у овом задатку, и његово тестирање

У реализованом SCL коду контролерског блока (FB1), заштита од негативног утицаја "препуњавања" интегратора (Anti-Windup) и сатурација коначног управљачког сигнала изведене су кроз двостепени алгоритам ограничавања унутрашњих променљивих. Директно ограничавање стања интегратора (State Saturation Anti-Windup), где се интегрална сума у сваком циклусу $T_s = 40$ [ms] ажурира на основу тренутне грешке према дискретној једначини:

$$u_integral_stat[k] = u_integral_stat[k - 1] + K_{i_step} * e[k]$$

Одмах након израчунавања, примењује се Anti-Windup логика која ограничава акумулирано стање интегратора на физичке границе [0.0, 125.0]. Ако акумулирана вредност пређе 125.0, она се закривљује (clamp-ује) на тачно 125.0. Ако падне испод 0.0, закривљује се на 0.0. Овим се спречава да интегратор настави са неограниченим акумулирањем грешке када је систем дуже време у засићењу, чиме се елиминише кашњење при повратку система у линеарни режим.

Коначни управљачки сигнал формира се као збир пропорционалне и ограничене интегралне компоненте ($u = u_p + u_i$). Након тога, над укупним сигналом се спроводи секундарна сатурација којом се гарантује да излаз регулатора према модели, односно објекту никада не пређе опсег [0.0, 125.0].

На дијаграму ARW теста (ARW_test_trace.csv) приказана је практична верификација заштитног Anti-Windup механизма заснованог на ограничавању стања интегратора (state clamping). Референтна вредност је постављена на високих $w[k] = 500$, док је излаз регулатора физички ограничен сатурацијом на $u_{max} = 125$, услед чега излаз објекта достиже свој максимум од $y[k] = 100$. Током периода трајања сатурације, грешка система остаје велика и позитивна ($e[k] = 400$), али стање интегратора је ограничено на 125 и не расте даље. У тренутку када задана вредност нагло падне на $w[k] = 0$, управљачки сигнал $u[k]$ и излаз објекта $y[k]$ моментално крећу да опадају без икаквог временског кашњења или "мртвог хода". Ова брза реакција је доказ да интегратор није претрпео "препуњавање" односно "windup" појаву, чиме је исправност имплементиране Anti-Windup логике у SCL коду практично и графички потврђена.

7. ПОРЕЂЕЊЕ РЕЗУЛТАТА: MATLAB/SIMULINK VS. PLC (TIA PORTAL)

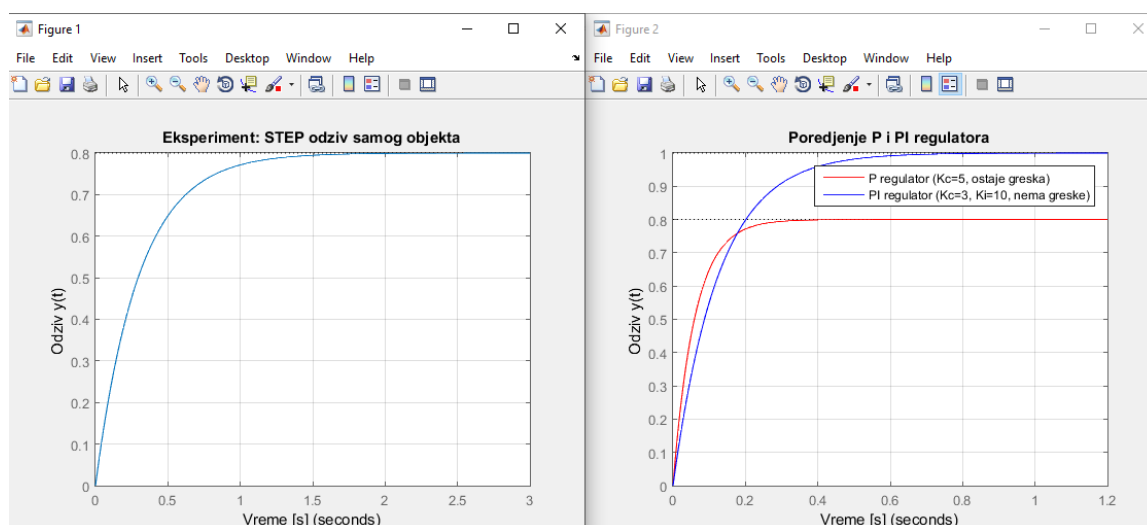
7.1. Симулација у MATLAB-у

Да би се извршила верификација тачности употребљеног SCL кода и дискретног модела у TIA Portal-у, спроведено је поређење добијених одзива са теоријским симулационим моделом изграђеним у програму MATLAB / Simulink.

У MATLAB-у је коришћена следећа скрипта како би се добили потребни графици:

```
K = 0.8;
tau = 0.3;
Ts = 0.04;
G = tf(K, [tau 1]);
figure(1);
step(G);
title('Eksperiment: STEP odziv samog objekta');
xlabel('Vreme [s]'); ylabel('Odziv y(t)');
grid on;
Kc_p = 5.0;
W_p = feedback(Kc_p * G, 1);
Kc_pi = 3.0;
Ti = 0.3;
C_pi = tf(Kc_pi * [Ti 1], [Ti 0]);
W_pi = feedback(C_pi * G, 1);
figure(2);
step(W_p, 'r', W_pi, 'b');
title('Poredjenje P i PI regulatora');
legend('P regulator (Kc=5, ostaje greska)', 'PI regulator (Kc=3, Ki=10, nema greske)');
xlabel('Vreme [s]'); ylabel('Odziv y(k)');
grid on;
```

Када покренемо скрипту добијамо следећа два дијаграма:



- Одзив самог објекта у отвореној петљи (figure(1))

Анализира се одзив објекта $G(s) = 0.8/(0.3s + 1)$ на одскачну побуду без присутности регулатора. Резултат који добијамо јесте да излаз $y(t)$ експоненцијално расте, и у стационарном стању достиже вредност једнаку статичком појачању објекта $y_{st} = K = 0.8$ (80%).

- Поређење П и ПИ регулатора у затвореној петљи (figure(2))

За П регулатор (црвена линија), при $K_{c_p} = 5.0$, преносна функција затвореног кола износи:

$$W_p(s) = \frac{5 * G(s)}{1 + 5 * G(s)} = \frac{5 * \frac{0.8}{0.3s + 1}}{1 + 5 * \frac{0.8}{0.3s + 1}} = \frac{4}{0.3s + 5} = \frac{0.8}{0.06s + 1}$$

Стационарна вредност је $y_{st} = 0.8$ (80%), што значи да у затвореном колу остаје статичка грешка од 20% ($e_{st} = 1 - 0.8 = 0.2$). Одзив је бржи у односу на отворену петљу, али грешка остаје.

За ПИ регулатор (плава линија): Применом ПИ регулатора са $K_{c_pi} = 3.0$ и $T_i = 0.3s$ остварује се поништавање доминантног пола објекта ($T_i = \tau = 0.3s$). Преносна функција регулатора је:

$$C_{pi}(s) = 3 * \frac{0.3s + 1}{0.3s}$$

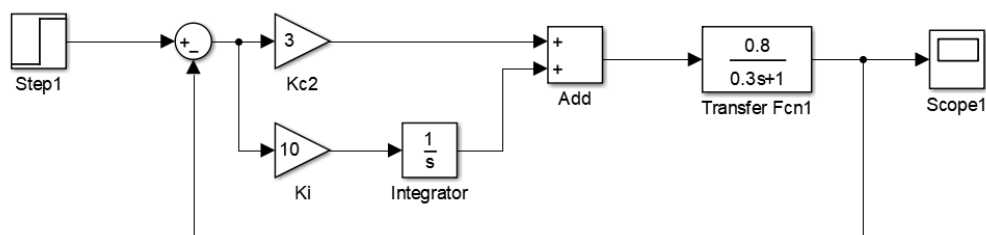
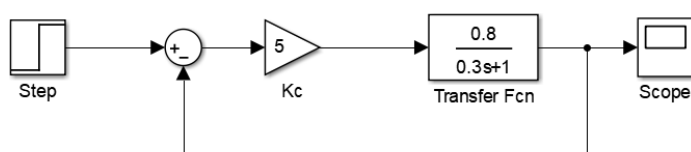
Преносна функција затвореног кола постаје:

$$W_{pi}(s) = \frac{C_{pi}(s) * G(s)}{1 + C_{pi}(s) * G(s)} = \frac{\frac{2.4}{0.3s}}{1 + \frac{2.4}{0.3s}} = \frac{2.4}{0.3s + 2.4} = \frac{1}{0.125s + 1}$$

Изаз $y(t)$ асимптотски достиже тачно 1.0 (100%), чиме је статичка грешка потпуно елиминисана ($e_{st} = 0$), без појаве прекорачења (препуњавања интегратора).

7.2. Симулација у SIMULINK-у

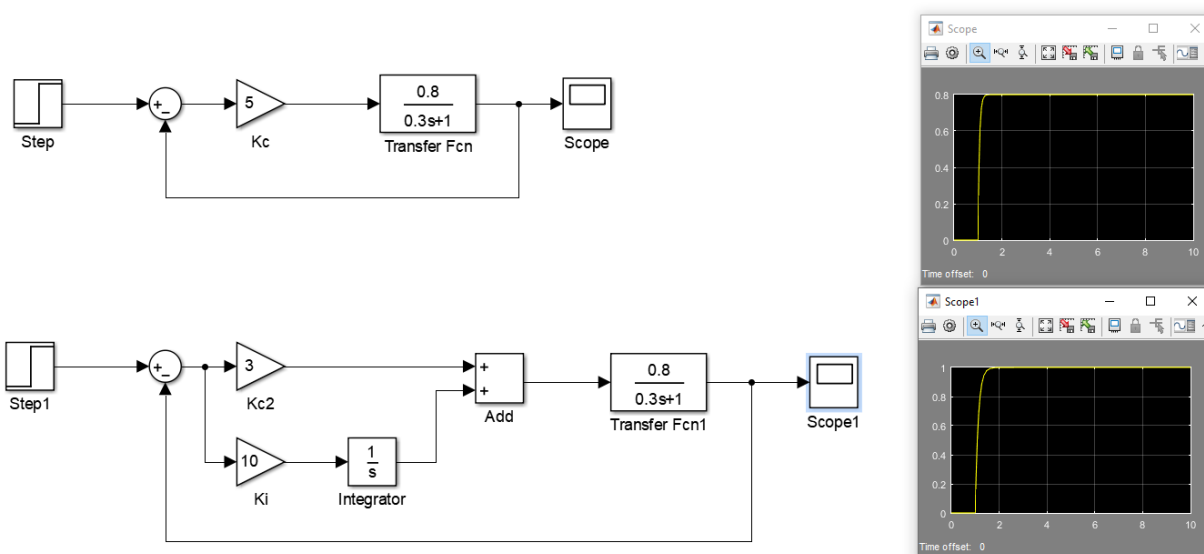
Модел се састоји од два одвојена подсистема који паралелно анализирају одзив објекта $G(s) = 0.8/(0.3s + 1)$ на јединичну одскачну побуду ($\text{Step} = 1.0$).



- Горњи подсистем – Чист П регулатор ($K_c = 5$):
 - Step: Улазни сигнал задате вредности $w(t) = 1.0$
 - Sum блок (+-): Формира сигнал грешке $e(t) = w(t) - y(t)$.
 - K_c (Gain блок = 5): Пропорционално појачање регулатора.
 - Transfer Fcn ($0.8/(0.3s+1)$): Континуални модел објекта управљања.
 - Scope: Блок за снимање и приказ одзива излазне величине $y(t)$.
- Доњи подсистем – Паралелни ПИ регулатор ($K_{c2} = 3$, $K_i = 10$):
 - Step1: Улазни сигнал задате вредности $w(t) = 1.0$
 - Sum блок (+-): Формира сигнал грешке $e(t) = w(t) - y(t)$
 - Пропорционална грана ($K_{c2} = 3$): Појачава сигнал грешке са коефицијентом $K_c = 3$
 - Интегрална грана ($K_i = 10$ и Integrator $1/s$): Формира интегрално дејство $K_i \int e(t) dt$, где је континуално интегрално појачање $K_i = 10.0 \text{ s}^{-1}$ (што директно одговара дискретном кораку $K_{i_step} = 0.4$ за $T = 0.04 \text{ s}$ јер је $K_i * T_s = 10 * 0.04 = 0.4$)

5. Add блок: Сабира пропорционалну и интегралну компоненту у укупни управљачки сигнал $u(t) = P(t) + I(t)$
6. Transfer Fcn1 ($0.8/(0.3s+1)$): Модел објекта
7. Scope1: Блок за праћење одзива ПИ регулатора.

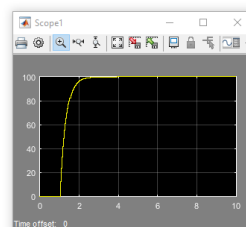
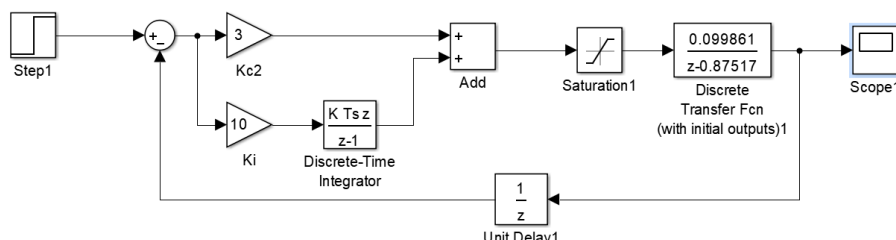
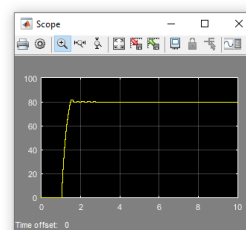
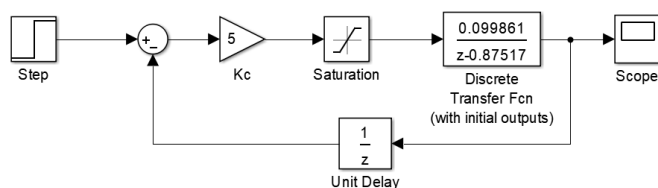
Дуплим кликом на два Scope-а добијамо следеће дијаграме:



Приказ са Scope-а (за П регулатор) потврђује горенаведене ставка, да систем са пропорционалним појачањем $K_c = 5$ достиже стационарну вредност $y_{st} = 0.8$. Грешка од 20% остаје трајно присутна јер управљачки сигнал опада како се излаз приближава задатој вредности, па нема елемента који би додатно погурао овај излаз до 1.0.

Приказ са Scope-а 1 (за ПИ регулатор) нам потврђује да интегратор у доњој грани успешно акумулира грешку кроз време све док је $e(t)$ различито од 0. Због тога излаз асимптотски али брзо достиже пуну вредност $y_{st} = 1.0$, а статичка грешка се потпуно елиминише ($e_{st} = 0$).

Ако сада Simulink графиком представимо наш сам систем, са периодом одабирања од 0.04 [s], и представимо преносну функцију преко блока за дискретно приказивање преносне функције у Simulink-у, добијамо дискретан симулинк модел који описује наш симулирани ПЛЦ систем првог реда. График и резултати се могу видети на следећој слици:



Да би Simulink модел био еквивалентан PLC алгоритму, модел садржи: Discrete Transfer Fcn $0.09986/(z - 0.87517)$ са $T_s = 0.04$ [s]; блок Unit Delay ($1/z$) у повратној грани (регулатор чита $y[k-1]$); за ПИ регулатор Discrete-Time Integrator (Accumulation method: Backward Euler, појачање $K_i T_s = 0.4$, границе излаза $[0, 125]$); и блок Saturation $[0, 125]$ на излазу регулатора.

У овим системима је такође убачен блок за сатурацију, који се налази у опсегу $[0, 125]$, као и у нашем коду. Овима можемо директно упоредити дискретан Simulink модел, са нашим симулираним ПЛЦ моделом.

7.3. Симулација дискретног модела еквивалентног PLC алгоритму

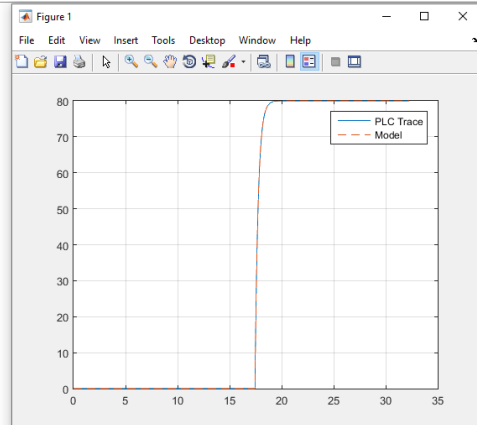
Континуални модели из претходних одељака (feedback без сатурације) нису еквивалентни PLC реализацији: PLC ради са периодом $T_s = 0.04$ [s], са сатурацијом $[0, 125]$, ограничењем стања интегратора и додатним тактом кашњења у повратној грани. Зато се верификација врши дискретним моделом истог алгоритма као SCL код и поређењем са подацима из TIA Portal Trace-а. Следећа скрипта учитава Trace .csv фајл (редослед колона: u, y, r), симулира исти алгоритам и рачуна одступање за сваки експеримент:

Испитни задатак из Аутоматског управљања

```

1 Ts = 0.04; a = 0.87517; b = 0.09986; umax = 125;
2
3 % --- избор фајла (или директно уписи име у fname) ---
4 [f, p] = uigetfile(['*.csv;*.txt','CSV / TXT'; '*.','Сви фајлови'], 'Изабери open loop CSV');
5 if isequal(f,0), disp('Није изабран фајл. '); return; end
6 fname = fullfile(p, f);
7 % fname = 'ovde_upisi_ime_fajla.csv'; % <- алтернатива, ако не желиш прозор
8
9 % --- учитавање (исто као у верзији која ради) ---
10 if exist('readmatrix','file')
11     T = readmatrix(fname);
12 else
13     T = csvread(fname, 1, 0); % 1 = прескочи заглавље
14 end
15 T = T(all(isfinite(T),2), :); % избаци заглавље/празне редове/NaN
16 assert(size(T,2) >= 3, 'CSV мора имати бар 3 колоне (u, y, z)');
17
18 u_plc = T(:,end-2); y_plc = T(:,end-1); z = T(:,end);
19 N = numel(z); t = (0:N-1)*Ts;
20
21 % --- отворена петља: u долази из снимка, нема регулатора ---
22 ystat = 0; ustat = 0; yout = 0;
23 y_sim = zeros(N,1); u_sim = zeros(N,1);
24
25 for k = 1:N
26     u = min(max(u_plc(k),0),umax);
27     ystat = a*ystat + b*ustat; % FB_Plant
28     yout = ystat; ustat = u;
29     y_sim(k) = yout; u_sim(k) = u;
30 end
31
32 fprintf('N = %d, max|dy| = %.4f, RMSE = %.4f\n', N, max(abs(y_sim-y_plc)), sqrt(mean((y_sim-y_plc).^2)));
33 plot(t,y_plc,t,y_sim,'--'); legend('PLC Trace','Model'); grid on

```

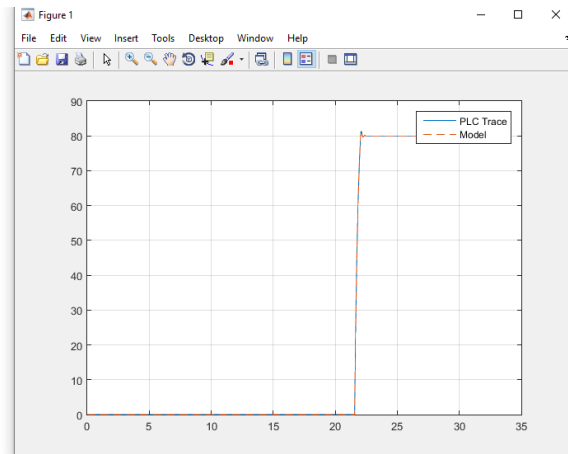


Симулинка симулација дискретног система са отвореном повратном спрегом

```

1 Ts = 0.04; a = 0.87517; b = 0.09986; umax = 125;
2 fname = 'closed_loop_P_trace.csv';
3
4 if exist('readmatrix','file')
5     T = readmatrix(fname);
6 else
7     T = csvread(fname, 1, 0);
8 end
9
10 T = T(all(isfinite(T),2), :);
11 assert(size(T,2) >= 3, 'CSV mora imati bar 3 kolone (u, y, z)');
12
13 u_plc = T(:,end-2); y_plc = T(:,end-1); z = T(:,end);
14 N = numel(z); t = (0:N-1)*Ts;
15
16 % --- regulator ---
17 Kc = 5; Ki = 0; pi_mode = false; arw = true; % за PI: Kc = 3, Ki = 10, pi_mode = true
18 ystat = 0; ustat = 0; yout = 0; ui = 0;
19 y_sim = zeros(N,1); u_sim = zeros(N,1);
20
21 for k = 1:N
22     e = r(k) - yout; % FBI vidi y iz prethodnog ciklusa
23     if pi_mode
24         ui = ui + Ki*Ts*e;
25         if arw, ui = min(max(ui,0),umax); end
26         u = Kc*e + ui;
27     else
28         u = Kc*e;
29     end
30     u = min(max(u,0),umax);
31     ystat = a*ystat + b*ustat; % FB_Plant
32     yout = ystat; ustat = u;
33     y_sim(k) = yout; u_sim(k) = u;
34 end
35
36 fprintf('N = %d, max|dy| = %.4f, RMSE = %.4f\n', N, max(abs(y_sim-y_plc)), sqrt(mean((y_sim-y_plc).^2)));
37 plot(t,y_plc,t,y_sim,'--'); legend('PLC Trace','Model'); grid on

```

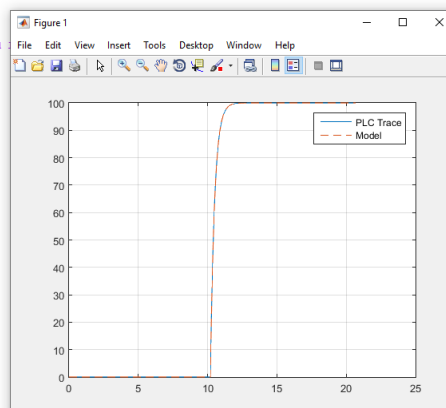


Симулинка симулација дискретног система са затвореном спрегом и П регулатором

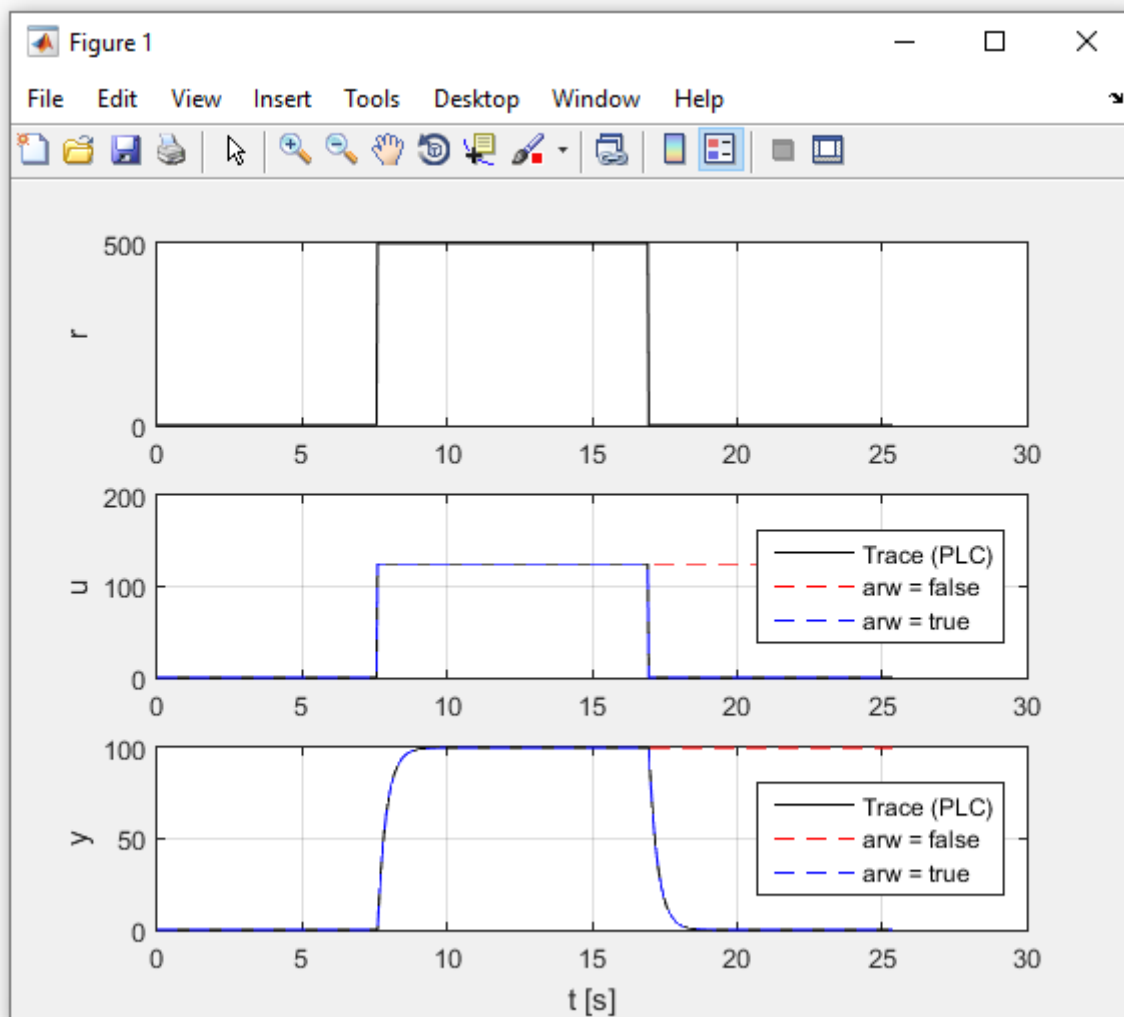
```

1 Ts = 0.04; a = 0.87517; b = 0.09986; umax = 125;
2
3 % --- избор фајла (или директно уписи име у fname) ---
4 [f, p] = uigetfile(['*.csv;*.txt','CSV / TXT'; '*.','Сви фајлови'], 'Изабери closed');
5 if isequal(f,0), disp('Није изабран фајл. '); return; end
6 fname = fullfile(p, f);
7 % fname = 'ovde_upisi_ime_fajla.csv'; % <- алтернатива, ако не желиш прозор
8
9 % --- учитавање ---
10 if exist('readmatrix','file')
11     T = readmatrix(fname);
12 else
13     T = csvread(fname, 1, 0); % 1 = прескочи заглавље
14 end
15 T = T(all(isfinite(T),2), :); % избаци заглавље/празне редове/NaN
16 assert(size(T,2) >= 3, 'CSV мора имати бар 3 колоне (u, y, z)');
17
18 u_plc = T(:,end-2); y_plc = T(:,end-1); z = T(:,end);
19 N = numel(z); t = (0:N-1)*Ts;
20
21 % --- PI регулатор ---
22 Kc = 3; Ki = 10; pi_mode = true; arw = true;
23 ystat = 0; ustat = 0; yout = 0; ui = 0;
24 y_sim = zeros(N,1); u_sim = zeros(N,1);
25
26 for k = 1:N
27     e = r(k) - yout; % FBI vidi y iz prethodnog ciklusa
28     if pi_mode
29         ui = ui + Ki*Ts*e;
30         if arw, ui = min(max(ui,0),umax); end
31         u = Kc*e + ui;
32     else
33         u = Kc*e;
34     end
35     u = min(max(u,0),umax);
36     ystat = a*ystat + b*ustat; % FB_Plant
37     yout = ystat; ustat = u;
38     y_sim(k) = yout; u_sim(k) = u;
39 end
40
41 fprintf('N = %d, max|dy| = %.4f, RMSE = %.4f\n', N, max(abs(y_sim-y_plc)), sqrt(mean((y_sim-y_plc).^2)));
42 plot(t,y_plc,t,y_sim,'--'); legend('PLC Trace','Model'); grid on

```



Симулинк симулација дискретног система са затвореном спрегом и ПИ регулатором



Графички приказ одзива система под утицајем ARW и без

Постављањем $arw = false$ и симулацијом ARW теста ($r = 500$, а затим $r = 0$) добија се одзив без ограничења интегратора. У том случају управљање након пада референце остаје на 125 знатно дуже (преко 25 [s]) за трајање засићења од око 9 [s]), док са ограничењем стања интегратора пада на 0 већ у првом такту.

Овим сликама доказујемо да се Trace фајлови експортирани из ТИА Портала потпуно поклапају са моделом симулираном у МАТЛАБ-у.

7.4. Читање log текстуалног фајла у MATLAB-у и исцртавање графика

Након што лоцирамо текстуални фајл на систему оператера(по дифолту то је фајл који креира сама TIA Portal,на локацији C:\Storage Card SD\Logs\Logging_file), потребно је да га убацимо у следећу MATLAB скрипту за читање података:

```

1 - [filename, filepath] = uigetfile('*.txt', 'Izaberite log fajl');
2
3 - if isequal(filename, 0)
4 -     disp('Izbor fajla je otkazan.');
```

return;

```
6 - end
7
8 - fullpath = fullfile(filepath, filename);
9
10 - fid = fopen(fullpath, 'r', 'n', 'UTF-16LE');
```

if fid == -1

```
12 -     fid = fopen(fullpath, 'r');
```

end

```
14
15 - raw = textscan(fid, '%q %q %f %f %f', 'HeaderLines', 1);
16 - fclose(fid);
17
18 - varNames = cellstr(raw{1});
19 - values = raw{3};
20
21 - validIdx = ~strncmp(varNames, '$', 1);
22 - varNames = varNames(validIdx);
23 - values = values(validIdx);
24
25 - uniqueVars = unique(varNames);
26
27 - T_hmi = 0.1;
28
29 - figure('Name', ['Prikaz signala: ' filename], 'NumberTitle', 'off');
```

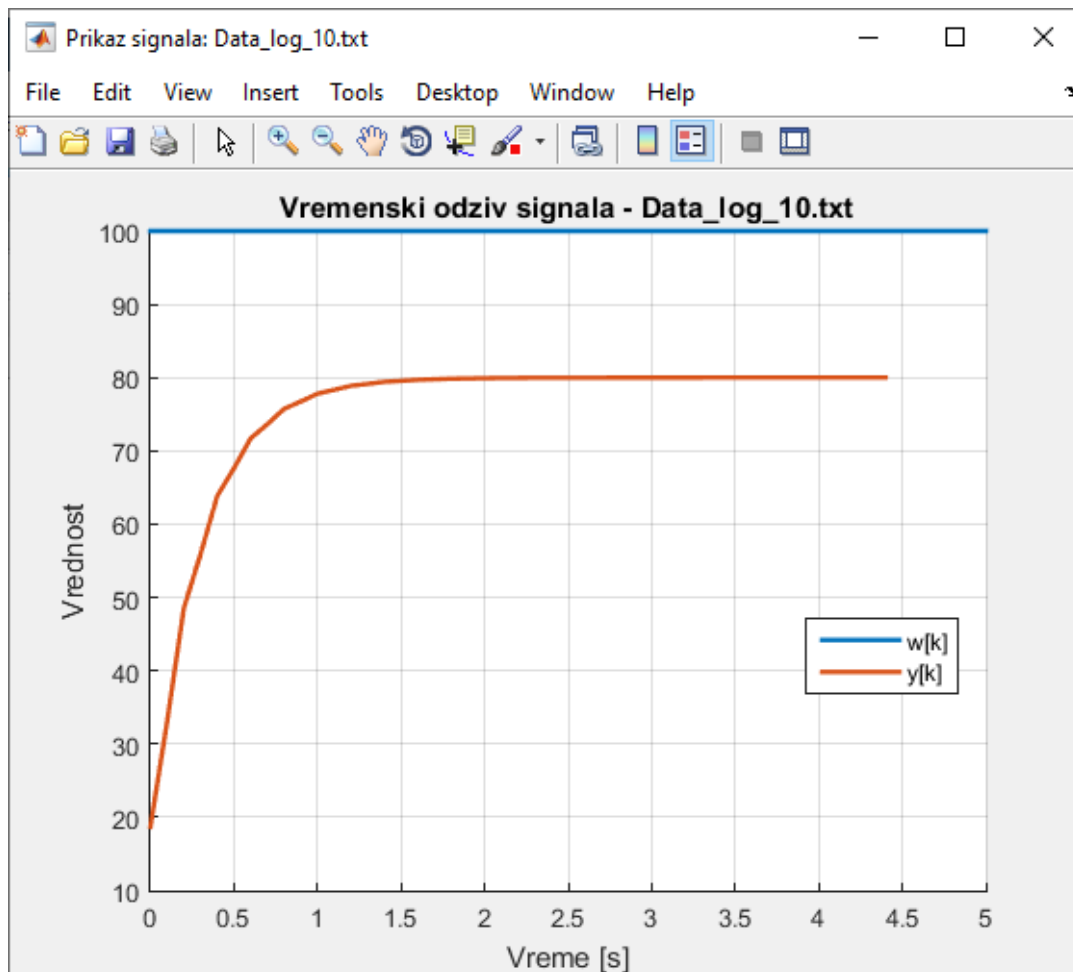
hold on;

```
31
32 - for i = 1:length(uniqueVars)
33 -     currentVar = uniqueVars{i};
34 -     idx = strcmp(varNames, currentVar);
35
36 -     y_var = values(idx);
37 -     t_var = (0:length(y_var)-1) * T_hmi;
38
39 -     if length(y_var) == 1
40 -         t_var = [0; 5.0];
41 -         y_var = [y_var; y_var];
42 -     end
43
44 -     plot(t_var, y_var, 'LineWidth', 1.8);
45 - end
46
47 - hold off;
48 - grid on;
49 - xlabel('Vreme [s]');
50 - ylabel('Vrednost');
```

title(['Vremenski odziv signala - ', strrep(filename, '_', '_')]);

```
52 - legend({'w[k]', 'y[k]'}, 'Location', 'best');
```

Уколико смо добро снимили податке из симулације, на графику који изађе појавиће се нешто овако:



Важно је напоменути да је ова провера оријентациона, HMI лог (Acquisition mode: On change) није једнолико одабран, па се не може користити за квантитативну верификацију. Поређење са моделом врши се искључиво на подацима TIA Portal Trace-а

Са овим смо завршили проверу резултата.

Сви фајлови, скрипте и зип-документи окачени су на Github репозиторијуму:

https://github.com/pointblank-100/Automatsko_upravljanje.git

8. Закључак

У овом раду успешно је пројектован, имплементиран и анализиран дискретни систем аутоматског управљања за систем првог реда преносне функције $G(s) = 0.8/(0.3s + 1)$. Симулација и валидација алгоритма извршене су коришћењем програмибилног логичког контролера Siemens S7-1214 DC/DC/DC у окружењу TIA Portal, као и рачунарских пакета MATLAB и Simulink. На основу спроведеног истраживања и експерименталних резултата, могу се дефинисати следећи кључни закључци.

Дискретизација континуалног модела ZOH поступком и његова имплементација у SCL језику унутар цикличног прекидног блока OB30 ($T_s = 40 \text{ ms}$) показала је велику прецизност. Одзиви добијени на PLC-у се поклапају са теоријским одзивима из MATLAB/Simulink симулација изведених над дискретним моделом истог алгоритма, као што је показано у одељку 7.3..

Примена чистог пропорционалног регулатора ($K_c = 5.0$) потврдила је основни теоријски недостатак овог типа управљања, карактеристичну статичку грешку. Иако повећање појачања K_c убрзава одзив и смањује грешку, она се не може потпуно елиминисати без интегралног дејства. Увођењем интегралне компоненте ($K_c = 3$, $K_i = 10$) статичка грешка је потпуно елиминисана ($e_{st} = 0$), а излазна величина је асимптотски достигла задату вредност без појава прекорачења или осцилација.

Имплементација ARW алгоритма са засићењем интегратора унутар SCL кода спречила је прекомерно нагомилавање интегралне суме током великих одступања, чиме је обезбеђена стабилност и адекватан прелазни процес при симулацији реалних ограничења излаза. Развијени HMI интерфејс (Root screen) са Trend View графиком омогућио је интуитивно праћење процеса у реалном времену, дијагностику и динамичку измену параметара помоћу параметарских поља на симулираном панелу. Овим је потврђено да креирана управљачка логика за Siemens S7-1200 представља поуздану, математички тачну и практично применљиву основу за управљање реалним индустријским процесима првог реда.